

Laporan Tugas Besar 1 IF2123 Aljabar Linier dan Geometri

Sistem Persamaan Linier, Determinan, Invers Matriks, dan Pemodelan Kurva



Disusun Oleh:

Rafel Dzinun Muhammad (13525063)

Kevin Sie (13525053)

Ahmad Boutros Fathir (13525002)

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika Komputasi
Institut Teknologi Bandung
2026**

Daftar Isi

1	Pendahuluan	3
1.1	Deskripsi Masalah	3
1.2	Latar Belakang	3
1.3	Tujuan	3
2	Dasar Teori	3
2.1	Sistem Persamaan Linier	3
2.2	Determinan Matriks	4
2.3	Matriks Balikan (Invers)	5
2.4	Interpolasi Polinomial	5
2.5	Interpolasi Spline Kubik (Natural Cubic Spline)	6
2.6	Regresi Spline Kubik	6
3	Implementasi	7
3.1	Struktur Kelas	7
3.2	Arsitektur Pustaka	7
3.3	Alur Program	7
3.4	Pemanfaatan Build Tool	8
4	Eksperimen	8
4.1	Determinan Matriks	8
4.1.1	Kasus Uji 5.1.1	8
4.1.2	Kasus Uji 5.1.2	9
4.1.3	Kasus Uji 5.1.3	10
4.2	Matriks Balikan (Invers)	11
4.2.1	Kasus Uji 5.2.1	11
4.2.2	Kasus Uji 5.2.2	13
4.3	Sistem Persamaan Linier	14
4.3.1	Kasus Uji 5.3.1	14
4.3.2	Kasus Uji 5.3.2	15
4.3.3	Kasus Uji 5.3.3	16
4.3.4	Kasus Uji 5.3.4	18
4.4	SPL Berbentuk Matriks Augmented	20
4.4.1	Kasus Uji 5.4.1	20
4.4.2	Kasus Uji 5.4.2	23
4.5	SPL Berbentuk Umum	24
4.5.1	Kasus Uji 5.5.1	24
4.5.2	Kasus Uji 5.5.2	25
4.6	Aplikasi Sistem Persamaan Linier	26
4.6.1	Kasus Uji 5.6	26
4.7	Interpolasi Polinomial	29
4.7.1	Kasus Uji 5.7 Studi Kasus 1	29
4.7.2	Kasus Uji 5.7 Studi Kasus 2	29
4.8	Natural Cubic Spline	30
4.8.1	Kasus Uji 5.8	30
4.9	Regresi Spline	31
4.9.1	Kasus Uji 5.9	31

4.10 Pemulihan Citra Digital (Image Inpainting - Bonus)	32
4.11 Performa pada Matriks Skala Maksimum	34
5 Penutup	34
5.1 Kesimpulan	34
5.2 Kelebihan dan Kekurangan	34
5.3 Saran	34
5.4 Refleksi	34

1 Pendahuluan

1.1 Deskripsi Masalah

Konsep fundamental aljabar linier seperti Sistem Persamaan Linier atau SPL, determinan, hingga matriks balikan merupakan pilar penting yang selalu hadir dalam berbagai komputasi keteknikan dan pengolahan data. Penyelesaian operasi ini secara manual mungkin masih praktis untuk matriks berukuran kecil seperti dimensi 3×3 , namun pendekatan tersebut menjadi lambat dan tidak efisien ketika ukuran matriks membengkak hingga puluhan atau ratusan baris. Kompleksitas serupa juga ditemukan dalam domain pemodelan data. Berbagai upaya untuk menarik kurva yang melewati sekumpulan titik observasi pada akhirnya selalu bermuara pada tahapan komputasi penyelesaian persamaan linier.

1.2 Latar Belakang

Berdasarkan spesifikasi tugas ini, seorang peneliti bernama Mitya membutuhkan perangkat lunak yang sanggup mengerjakan semua perhitungan matematis tersebut secara otomatis. Oleh karena itu, kelompok kami ditugaskan untuk merancang dan membangun pustaka aljabar linier sekaligus program aplikasinya menggunakan bahasa pemrograman Java murni. Kami diuji untuk mampu menerjemahkan logika matematika menjadi baris kode yang fungsional, tangguh dalam menangani masukan galat, serta tetap cepat saat menghadapi matriks berskala masif tanpa memanfaatkan pustaka bantuan luar sama sekali.

1.3 Tujuan

Tujuan utama pengerjaan tugas besar ini adalah mengimplementasikan algoritma penyelesaian sistem persamaan linier, pencarian determinan, matriks balikan, serta merancang fungsionalitas pemodelan kurva yang meliputi interpolasi polinomial, spline kubik, regresi spline, dan pemrosesan pemulihan citra. Seluruh fungsionalitas tersebut dibungkus ke dalam sebuah pustaka modular yang dijalankan melalui antarmuka baris perintah yang mudah digunakan.

2 Dasar Teori

2.1 Sistem Persamaan Linier

Sistem Persamaan Linier (SPL) adalah himpunan berhingga dari persamaan-persamaan linier dengan variabel-variabel yang sama. Secara umum, sistem dengan m persamaan dan n variabel dapat direpresentasikan dalam bentuk persamaan aljabar matriks $Ax = b$, di mana A adalah matriks koefisien berukuran $m \times n$, x adalah vektor variabel kolom, dan b adalah vektor konstanta hasil. Untuk mempermudah komputasi, matriks ini sering digabungkan menjadi matriks *augmented* berbentuk $[A|b]$. Solusi dari sebuah SPL selalu jatuh ke dalam salah satu dari tiga kategori matematis: memiliki solusi tunggal, memiliki banyak solusi (parametrik), atau tidak memiliki solusi sama sekali (inkonsisten). Terdapat empat metode utama yang diimplementasikan untuk menyelesaikan masalah ini:

- **Eliminasi Gauss:** Metode ini berfokus pada pengubahan matriks *augmented* menjadi bentuk eselon baris (*Row Echelon Form*) melalui serangkaian Operasi Baris Elementer (OBE). Pada bentuk ini, elemen di bawah setiap pivot bernilai nol dan setiap pivot bernilai satu. Setelah bentuk eselon baris tercapai, nilai masing-masing variabel dicari dari bawah ke atas menggunakan teknik substitusi mundur (*backward substitution*).
- **Eliminasi Gauss-Jordan:** Merupakan bentuk lanjutan dari eliminasi Gauss. Operasi baris elementer diteruskan hingga matriks mencapai bentuk eselon baris tereduksi (*Reduced Row Echelon Form*). Pada kondisi ini, elemen di atas maupun di bawah pivot utama semuanya bernilai nol, sehingga matriks koefisien berubah menjadi matriks identitas dan solusi untuk masing-masing variabel dapat langsung dibaca secara sejajar pada kolom konstanta di ruas paling kanan.
- **Partial Pivoting (Tata-ancang Parsial):** Mengingat memori komputer menyimpan bilangan real dengan presisi *floating point* yang terbatas, pembagian dengan angka pivot yang sangat kecil dapat memperbesar galat pembulatan secara drastis. Oleh karena itu, pada eliminasi Gauss dan Gauss-Jordan diterapkan teknik *partial pivoting*. Algoritma ini memindai setiap kolom untuk mencari nilai mutlak terbesar, lalu menukar baris tersebut ke posisi pivot teratas guna meminimalisasi galat dan mencegah terjadinya pembagian dengan nol (*division by zero*).
- **Metode Matriks Balikan:** Metode ini bertumpu pada sifat invers matriks dan hanya berlaku jika matriks koefisien A berbentuk persegi ($n \times n$) serta bersifat non-singular ($\det(A) \neq 0$). Solusi dari sistem persamaan linier $Ax = b$ dapat secara langsung dihitung dengan mengalikan matriks balikan koefisien dengan vektor konstanta hasil, yaitu $x = A^{-1}b$.
- **Kaidah Cramer:** Sama seperti metode matriks balikan, kaidah Cramer mensyaratkan matriks A berbentuk persegi dan determinannya tidak bernilai nol. Perhitungan solusi setiap variabel dilakukan dengan membagi determinan matriks A_i dengan nilai determinan matriks awal A . Matriks A_i dibentuk dengan mengganti seluruh elemen pada kolom ke- i dari matriks A dengan vektor konstanta b . Secara matematis dirumuskan sebagai:

$$x_i = \frac{\det(A_i)}{\det(A)}$$

2.2 Determinan Matriks

Determinan adalah suatu nilai skalar unik yang hanya dapat dikomputasi dari sebuah matriks persegi berordo $n \times n$. Nilai determinan memegang peranan vital dalam aljabar linier karena menjadi indikator utama apakah suatu matriks memiliki balikan (non-singular, $\det(A) \neq 0$) atau tidak (singular, $\det(A) = 0$). Sifat-sifat dasar determinan meliputi determinan matriks transpos sama dengan matriks asalnya ($\det(A^T) = \det(A)$), serta penukaran dua baris matriks akan membalikkan tanda determinannya. Dua pendekatan utama digunakan dalam menghitung determinan:

- **Metode Reduksi Baris (OBE):** Merupakan metode yang sangat efisien untuk matriks berskala masif dengan kompleksitas waktu $O(n^3)$. Matriks direduksi

menggunakan operasi baris elementer hingga membentuk matriks segitiga atas. Pada bentuk ini, determinan matriks adalah hasil kali dari semua elemen pada diagonal utamanya, dikalikan dengan $(-1)^k$, di mana k adalah jumlah operasi penukaran baris (*row swapping*) yang terjadi selama proses reduksi.

- **Metode Ekspansi Kofaktor:** Metode ini menggunakan pendekatan dekomposisi rekursif untuk memecah matriks berukuran besar menjadi sub-matriks minor yang lebih kecil. Ekspansi kofaktor sepanjang baris ke- i dirumuskan sebagai:

$$\det(A) = \sum_{j=1}^n a_{ij} C_{ij}$$

Nilai kofaktor C_{ij} didapatkan dari persamaan $C_{ij} = (-1)^{i+j} M_{ij}$, di mana M_{ij} adalah determinan minor matriks yang dihasilkan setelah menghapus baris ke- i dan kolom ke- j . Karena kompleksitas waktunya mencapai $O(n!)$, metode ini mengoptimalkan eksekusinya dengan selalu memilih baris atau kolom yang memiliki elemen angka nol terbanyak sebagai acuan ekspansi.

2.3 Matriks Balikan (Invers)

Matriks balikan, dilambangkan dengan A^{-1} , adalah matriks yang secara khusus memenuhi identitas perkalian $A \cdot A^{-1} = A^{-1} \cdot A = I$, di mana I adalah matriks identitas seukuran matriks A . Matriks balikan juga memiliki sifat-sifat matematis turunan, seperti $(A^{-1})^{-1} = A$ dan $(AB)^{-1} = B^{-1}A^{-1}$. Pencarian invers dalam pustaka ini difasilitasi melalui dua algoritma:

- **Metode Augmentasi (Eliminasi Gauss-Jordan):** Metode komputasi numerik ini menjejerkan matriks asal A dengan matriks identitas I di dalam satu bingkai matriks *augmented* berbentuk $[A|I]$. Seluruh matriks tersebut kemudian direduksi menggunakan operasi baris elementer secara serentak. Jika ruas kiri matriks berhasil berubah menjadi matriks identitas murni, maka ruas kanan matriks yang semula identitas telah bertransformasi menjadi matriks balikan A^{-1} .
- **Metode Adjoin:** Pencarian balikan dengan pendekatan analitik ini memanfaatkan matriks kofaktor. Mula-mula, matriks kofaktor C disusun dari perhitungan determinan minor pada setiap elemen. Transpos dari matriks kofaktor ini disebut sebagai matriks adjoin ($Adj(A) = C^T$). Matriks balikan akhirnya didapatkan dengan mengalikan matriks adjoin tersebut dengan skalar invers dari determinan matriks asalnya:

$$A^{-1} = \frac{1}{\det(A)} Adj(A)$$

2.4 Interpolasi Polinomial

Interpolasi polinomial adalah metode untuk memodelkan fungsi yang tidak diketahui asalnya menjadi sebuah polinomial yang secara presisi melintasi sekumpulan titik data diskrit. Jika diberikan sekumpulan data yang terdiri dari $n + 1$ titik sampel berbentuk $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ yang masing-masing unik nilainya, maka akan dapat dikonstruksi tepat satu buah polinomial dengan derajat tertinggi n yang berbentuk:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

Pencarian parameter koefisien $a_0, a_1, \dots, a_n$ dilakukan dengan menyubstitusikan nilai titik absis dan ordinat masukan ke dalam bentuk deret pangkat berurutan, sehingga membentuk Sistem Persamaan Linier yang disebut dengan matriks Vandermonde:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Matriks Vandermonde di atas lalu diselesaikan dengan algoritma Eliminasi Gauss untuk memperoleh koefisien polinomial pembentuk kurva interpolasinya.

2.5 Interpolasi Spline Kubik (Natural Cubic Spline)

Metode interpolasi polinomial berderajat tinggi cenderung memicu osilasi atau fluktuasi ekstrem di area ujung kurva (Fenomena Runge). Interpolasi spline mengatasi limitasi tersebut dengan membagi rentang domain menjadi beberapa interval antar-titik data dan menggunakan persamaan polinomial berderajat rendah (kubik) yang disambungkan potong demi potong. Fungsi kubik pada setiap sub-interval $[x_i, x_{i+1}]$ didefinisikan dengan:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Untuk menjamin agar setiap sambungan kurva terasa mulus secara visual, turunan pertama dan kedua antar-fungsi segmen di titik sambungan diwajibkan harus berkesinambungan. Syarat kelancaran ini, dikombinasikan dengan definisi lebar interval $h_i = x_{i+1} - x_i$ dan nilai turunan kedua $M_i = S''(x_i)$, akan melahirkan persamaan tridiagonal raksasa:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right)$$

Persamaan tridiagonal ini memuat *syarat batas natural* yang memaksa nilai turunan kedua pada titik batas kurva paling ujung bernilai mutlak nol ($M_0 = M_n = 0$). Penyelesaian matriks ini kemudian membuahkan serangkaian nilai parameter untuk menghitung koefisien fungsi kubik a_i, b_i, c_i , dan d_i pada masing-masing segmen.

2.6 Regresi Spline Kubik

Berbeda dengan interpolasi murni yang memaksa kurva memotong seluruh titik, regresi dirancang untuk menelusuri tren garis tengah pada sebaran titik observasi dalam jumlah besar untuk meminimalkan anomali data (kuadrat terkecil residu). Regresi spline menggunakan basis fungsi tersegmen yang disebut dengan *basis daya terpancung* (truncated power basis), disimbolkan dengan $(x - \xi_k)_+^3$, di mana ξ_k merupakan posisi titik simpul peralihan (knot). Sifat terpancung menjamin kurva tetap stabil dengan nilai kubik berlaku jika $x > \xi_k$ dan bernilai 0 apabila sebaliknya. Model utuh regresi spline ini direpresentasikan oleh fungsi:

$$\hat{y}(x) = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \sum_{k=1}^K \beta_{3+k}(x - \xi_k)_+^3$$

Matriks desain X akan dibentuk dari proyeksi nilai x masukan ke basis-basis pemotong tersebut. Proses pencarian himpunan bobot parameter β terbaik dilakukan dengan mentransformasikan masalah ini ke ruang Persamaan Normal:

$$(X^T X)\beta = X^T Y$$

Model persamaan ini secara numerik dikomputasi menggunakan perkalian matriks standar yang kemudian dieksekusi dengan teknik eliminasi matriks.

3 Implementasi

3.1 Struktur Kelas

Program dikembangkan dengan paradigma berorientasi objek dalam bahasa Java murni. Kelas dasar `Matrix.java` bertanggung jawab penuh atas penyimpanan data dalam bentuk himpunan dimensi ganda primitif serta menyediakan fungsi operasi aritmatika dan manipulasi baris elementer. Logika penyelesaian lanjutan dipisah ke dalam modul-modul mandiri, antara lain `SPL.java` untuk menangani komputasi eliminasi dengan fitur penukaran baris parsial, `Determinan.java` dan `Invers.java` untuk komputasi matriks inti, serta `InterpolasiPolinomial.java`, `SplineKubik.java`, dan `RegresiSpline.java` untuk pembentukan matriks permodelan kurva. Operasi manipulasi warna dan penerapan algoritma lelaran iteratif untuk pemulihan foto ditangani oleh kelas `ImageInpainting.java`. Interaksi dan pemrosesan masukan data dari pengguna difasilitasi sepenuhnya oleh kelas `IOHandler.java`.

3.2 Arsitektur Pustaka

Seluruh modul dirancang sedemikian rupa agar logika komputasi murni terpisah dari lapisan antarmuka interaksi pengguna. Kelas `Matrix.java` berkedudukan sebagai tulang punggung struktur data yang dipanggil oleh semua modul analisis teoritis. Komputasi matriks yang spesifik seperti pencarian kofaktor akan memanggil fungsi bawaan pustaka matriks, sementara pencarian matriks bentuk eselon baris akan mendelegasikan perhitungannya pada kelas fungsi persamaan linier terkait.

3.3 Alur Program

Sistem antarmuka baris perintah dirancang untuk memberikan pengalaman navigasi yang intuitif. Pengguna memulai interaksi melalui menu utama yang merangkum seluruh fitur aplikasi. Saat pengguna memilih salah satu modul penyelesaian aljabar, program akan meminta pengguna memilih metode komputasi yang diinginkan beserta format masukannya, baik berupa ketikan langsung melalui papan ketik maupun pemuatan data dari berkas teks. Pustaka `IOHandler` kemudian melakukan validasi data dan memindahkannya ke dalam memori. Setelah modul berhasil melakukan perhitungan, sistem akan mencetak hasil luaran lengkap beserta jejak langkah manipulasi operasi barisnya ke layar terminal dengan tawaran untuk menyimpan riwayat tersebut ke dalam sebuah berkas teks baru.

3.4 Pemanfaatan Build Tool

Proyek ini dikonfigurasi menggunakan alat manajemen perangkat lunak Maven. Penggunaan Maven sangat mempermudah proses kompilasi kode dan penyatuan struktur proyek ke dalam satu distribusi paket. Dengan mengeksekusi instruksi perakitan Maven, seluruh kode sumber, konfigurasi paket, dan penanganan galat akan dikompilasi menjadi satu berkas Java Archive (.jar) yang bisa dijalankan secara mandiri pada berbagai mesin eksekusi Java.

4 Eksperimen

Bagian ini mendokumentasikan hasil eksekusi program terhadap kasus uji yang tercantum pada dokumen spesifikasi tugas besar. Setiap pengujian memuat data masukan, tangkapan layar (*screenshot*) luaran dari terminal, serta analisis komprehensif terkait kecocokan solusi eksak.

4.1 Determinan Matriks

4.1.1 Kasus Uji 5.1.1

Masukan: Matriks persegi berukuran 4x4.

$$A = \begin{bmatrix} 0 & 2 & -1 & 3 \\ 4 & 1 & 2 & -2 \\ -2 & 3 & 0 & 1 \\ 1 & -1 & 5 & 2 \end{bmatrix}$$

Keluaran: Berikut adalah hasil komputasi menggunakan Metode Reduksi Baris:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 2

--- Determinan Matriks ---
1. Metode Reduksi Baris
2. Metode Ekspansi Kofaktor
Pilih metode (1-2): 1
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/det_kasus_5_1_1.txt

Langkah-langkah:
Matriks awal:
  0.000  2.000  -1.000  3.000
  4.000  1.000  2.000  -2.000
 -2.000  3.000  0.000  1.000
  1.000 -1.000  5.000  2.000
R1 <-> R2 (tanda determinan dikali -1)
  4.000  1.000  2.000  -2.000
  0.000  2.000  -1.000  3.000
 -2.000  3.000  0.000  1.000
  1.000 -1.000  5.000  2.000
R3 <- R3 + 0.5*R1
  4.000  1.000  2.000  -2.000
  0.000  2.000  -1.000  3.000
  0.000  3.500  1.000  0.000
  1.000 -1.000  5.000  2.000
R4 <- R4 - 0.25*R1
  4.000  1.000  2.000  -2.000
  0.000  2.000  -1.000  3.000
  0.000  3.500  1.000  0.000
  0.000 -1.250  4.500  2.500
R2 <-> R3 (tanda determinan dikali -1)
  4.000  1.000  2.000  -2.000
  0.000  3.500  1.000  0.000
  0.000  2.000  -1.000  3.000
  0.000 -1.250  4.500  2.500
R3 <- R3 - 0.571*R2
  4.000  1.000  2.000  -2.000
  0.000  3.500  1.000  0.000
  0.000  0.000  -1.571  3.000
  0.000 -1.250  4.500  2.500
R4 <- R4 + 0.357*R2
  4.000  1.000  2.000  -2.000
  0.000  3.500  1.000  0.000
  0.000  0.000  -1.571  3.000
  0.000  0.000  4.857  2.500
R3 <-> R4 (tanda determinan dikali -1)
  4.000  1.000  2.000  -2.000
  0.000  3.500  1.000  0.000
  0.000  0.000  4.857  2.500
  0.000  0.000  -1.571  3.000
R4 <- R4 + 0.324*R3
  4.000  1.000  2.000  -2.000
  0.000  3.500  1.000  0.000
  0.000  0.000  4.857  2.500
  0.000  0.000  0.000  3.809

Matriks sudah berbentuk segitiga atas. Jumlah pertukaran baris = 3
det(A) = (-1) x 4 x 3.5 x 4.857 x 3.809 = -259
Determinan = -259

```

Gambar 1: Hasil Eksekusi Kasus Uji 5.1.1 (Metode Reduksi Baris)

Analisis: Hasil keluaran program terbukti sangat akurat dengan luaran akhir determinan bernilai -259. Berdasarkan perhitungan eksak menggunakan cara manual, nilai determinan matriks tersebut memang bernilai negatif 259. Program juga sukses menangani keberadaan angka nol pada elemen baris pertama kolom pertama tanpa mengalami galat pembagian berkat implementasi algoritma penukaran baris secara otomatis.

4.1.2 Kasus Uji 5.1.2

Masukan: Matriks persegi berukuran 5x5.

$$B = \begin{bmatrix} 2 & -1 & 3 & 0 & 4 \\ 1 & 2 & -2 & 5 & -1 \\ 3 & 0 & 1 & 2 & 0 \\ 0 & 4 & -1 & 3 & 2 \\ 5 & 0 & 4 & 2 & 4 \end{bmatrix}$$

Keluaran: Berikut adalah hasil komputasi menggunakan Metode Ekspansi Kofaktor:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 2

--- Determinan Matriks ---
1. Metode Reduksi Baris
2. Metode Ekspansi Kofaktor
Pilih metode (1-2): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/det_kasus_5_1_2.txt

Langkah-langkah:
Ekspansi kofaktor sepanjang baris 1 dari matriks:
2.000 -1.000 3.000 0.000 4.000
1.000 2.000 -2.000 5.000 -1.000
3.000 1.000 1.000 2.000 0.000
0.000 4.000 -1.000 3.000 2.000
5.000 0.000 4.000 2.000 4.000

Suku ke-1: + a11 x det(M11)
Minor M11 =
2.000 -2.000 5.000 -1.000
1.000 1.000 2.000 0.000
4.000 -1.000 3.000 2.000
0.000 4.000 2.000 4.000
det(M11) = -114
Suku = + (2) x (-114) = -228

Suku ke-2: - a12 x det(M12)
Minor M12 =
1.000 -2.000 5.000 -1.000
3.000 1.000 2.000 0.000
0.000 -1.000 3.000 2.000
5.000 4.000 2.000 4.000
det(M12) = -27
Suku = - (-1) x (-27) = -27

Suku ke-3: + a13 x det(M13)
Minor M13 =
1.000 2.000 5.000 -1.000
3.000 1.000 2.000 0.000
0.000 4.000 3.000 2.000
5.000 0.000 2.000 4.000
det(M13) = 177
Suku = + (3) x (177) = 531

Suku ke-4: a14 = 0, suku bernilai 0 (dilewati)

Suku ke-5: + a15 x det(M15)
Minor M15 =
1.000 2.000 -2.000 5.000
3.000 1.000 1.000 2.000
0.000 4.000 -1.000 3.000
5.000 0.000 4.000 2.000
det(M15) = -69
Suku = + (4) x (-69) = -276

det(A) = jumlah semua suku = 0
Determinan = 0

```

Gambar 2: Hasil Eksekusi Kasus Uji 5.1.2 (Metode Ekspansi Kofaktor)

Analisis: Program berhasil menguraikan matriks berdimensi lima ini ke dalam bentuk sub-matriks minor yang lebih kecil secara rekursif. Hasil hitungan akhir menunjukkan bahwa nilai determinan matriks ini adalah 0. Hasil ini akurat dan menandakan bahwa matriks tersebut bersifat singular, di mana terdapat ketergantungan linier antar baris pembentuknya.

4.1.3 Kasus Uji 5.1.3

Masukan: Matriks berukuran 4x4 dengan elemen pecahan yang dikonversi menjadi format desimal.

$$C = \begin{bmatrix} 0.5 & 1 & 0 & 0.5 \\ 0.5 & 0.75 & 0.25 & 0.25 \\ -0.025 & -0.025 & 0.025 & 0.025 \\ 0.025 & -0.075 & 0.075 & 0.075 \end{bmatrix}$$

Keluaran: Berikut adalah hasil komputasi menggunakan Metode Ekspansi Kofaktor:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 2

--- Determinan Matriks ---
1. Metode Reduksi Baris
2. Metode Ekspansi Kofaktor
Pilih metode (1-2): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/det_kasus_5_1_3.txt

Langkah-langkah:
Ekspansi kofaktor sepanjang baris 1 dari matriks:
0.500  1.000  0.000  0.500
0.500  1.250  0.250  0.500
-0.100 -0.400  0.200 -0.300
0.600  1.250  0.450  0.600

Suku ke-1: + a11 x det(M11)
Minor M11 =
1.250  0.250  0.500
-0.400  0.200 -0.300
1.250  0.450  0.600
det(M11) = 0.07
Suku = + (0.5) x (0.07) = 0.035

Suku ke-2: - a12 x det(M12)
Minor M12 =
0.500  0.250  0.500
-0.100  0.200 -0.300
0.600  0.450  0.600
det(M12) = 0.015
Suku = - (1) x (0.015) = -0.015

Suku ke-3: a13 = 0, suku bernilai 0 (dilewati)

Suku ke-4: - a14 x det(M14)
Minor M14 =
0.500  1.250  0.250
-0.100 -0.400  0.200
0.600  1.250  0.450
det(M14) = 0.02
Suku = - (0.5) x (0.02) = -0.01

det(A) = jumlah semua suku = 0.01
Determinan = 0.01

```

Gambar 3: Hasil Eksekusi Kasus Uji 5.1.3 (Matriks Pecahan)

Analisis: Komputasi matriks dengan elemen desimal ini dieksekusi dengan baik oleh program, menghasilkan nilai determinan akhir sebesar 0.01. Hasil ini terbukti sangat presisi dan ekuivalen dengan komputasi penyelesaian matriks identik menggunakan kakas standar. Hal ini membuktikan bahwa program mampu menangani komputasi bilangan bertipe *floating-point* secara rekursif tanpa kehilangan akurasi yang signifikan akibat galat pembulatan.

4.2 Matriks Balikan (Invers)

4.2.1 Kasus Uji 5.2.1

Masukan: Matriks persegi berukuran 4x4 yang bersifat simetris.

$$C = \begin{bmatrix} 0 & 1 & 2 & -1 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ -1 & 2 & 1 & 0 \end{bmatrix}$$

Keluaran: Berikut adalah langkah-langkah dan hasil pencarian invers menggunakan Metode Augmentasi:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image inpainting
8. Keluar
Pilih menu (1-8): 3

--- Matriks Balikan (Invers) ---
1. Metode Augmentasi
2. Metode Adjoin
Pilih metode (1-2): 1
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/inverse_kasus_5_2_1.txt

Langkah-langkah:
Matriks augmentasi [A | I]:
0.000 1.000 2.000 -1.000 1.000 0.000 0.000 0.000
1.000 0.000 1.000 2.000 0.000 1.000 0.000 0.000
2.000 1.000 0.000 1.000 0.000 0.000 1.000 0.000
-1.000 2.000 1.000 0.000 0.000 0.000 0.000 1.000
R1 <-> R3
2.000 1.000 0.000 1.000 0.000 0.000 1.000 0.000
1.000 0.000 1.000 2.000 0.000 1.000 0.000 0.000
0.000 1.000 2.000 -1.000 1.000 0.000 0.000 0.000
-1.000 2.000 1.000 0.000 0.000 0.000 0.000 1.000
R1 <- R1 / 2
1.000 0.500 0.000 0.500 0.000 0.000 0.500 0.000
1.000 0.000 1.000 2.000 0.000 1.000 0.000 0.000
0.000 1.000 2.000 -1.000 1.000 0.000 0.000 0.000
-1.000 2.000 1.000 0.000 0.000 0.000 0.000 1.000
R3 <- R3 / 1.6
1.000 0.000 -0.200 0.400 0.000 0.000 0.400 -0.200
0.000 1.000 0.400 0.200 0.000 0.000 0.200 0.400
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 1.200 1.600 0.000 1.000 -0.400 0.200
R1 <- R1 + 0.2*R3
1.000 0.000 0.000 0.250 0.125 0.000 0.375 -0.250
0.000 1.000 0.400 0.200 0.000 0.000 0.200 0.400
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 1.200 1.600 0.000 1.000 -0.400 0.200
R2 <- R2 - 0.4*R3
1.000 0.000 0.000 0.250 0.125 0.000 0.375 -0.250
0.000 1.000 0.000 0.500 -0.250 0.000 0.250 0.500
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 1.200 1.600 0.000 1.000 -0.400 0.200
R4 <- R4 - 1.2*R3
1.000 0.000 0.000 0.250 0.125 0.000 0.375 -0.250
0.000 1.000 0.000 0.500 -0.250 0.000 0.250 0.500
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 0.000 2.500 -0.750 1.000 -0.250 0.500
R4 <- R4 / 2.5
1.000 0.000 0.000 0.250 0.125 0.000 0.375 -0.250
0.000 1.000 0.000 0.500 -0.250 0.000 0.250 0.500
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 0.000 1.000 -0.300 0.400 -0.100 0.200
R1 <- R1 - 0.25*R4
1.000 0.000 0.000 0.000 0.200 -0.100 0.400 -0.300
0.000 1.000 0.000 0.500 -0.250 0.000 0.250 0.500
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 0.000 1.000 -0.300 0.400 -0.100 0.200
R2 <- R2 - 0.5*R4
1.000 0.000 0.000 0.000 0.200 -0.100 0.400 -0.300
0.000 1.000 0.000 0.000 -0.100 -0.200 0.300 0.400
0.000 0.000 1.000 -0.750 0.625 0.000 -0.125 -0.250
0.000 0.000 0.000 1.000 -0.300 0.400 -0.100 0.200
R2 <- R2 - 1*R1
0.000 -0.500 0.000 0.000 0.000 0.000 -0.500 0.000
0.000 1.000 0.000 0.000 0.000 0.000 0.000 0.000
0.000 0.000 1.000 0.000 0.000 0.000 0.000 0.000
0.000 2.500 1.000 0.000 0.000 0.000 0.500 1.000
R2 <-> R4
1.000 0.500 0.000 0.500 0.000 0.000 0.500 0.000
0.000 2.500 1.000 0.500 0.000 0.000 0.500 1.000
0.000 1.000 2.000 -1.000 1.000 0.000 0.000 0.000
0.000 -0.500 1.000 1.500 0.000 1.000 -0.500 0.000
R2 <- R2 / 2.5
1.000 0.500 0.000 0.500 0.000 0.000 0.500 0.000
0.000 1.000 0.400 0.200 0.000 0.000 0.200 0.400
0.000 1.000 2.000 -1.000 1.000 0.000 0.000 0.000
0.000 -0.500 1.000 1.500 0.000 1.000 -0.500 0.000
R1 <- R1 - 0.5*R2
1.000 0.000 -0.200 0.400 0.000 0.000 0.400 -0.200
0.000 1.000 0.400 0.200 0.000 0.000 0.200 0.400
0.000 1.000 2.000 -1.000 1.000 0.000 0.000 0.000
0.000 -0.500 1.000 1.500 0.000 1.000 -0.500 0.000
R3 <- R3 - 1*R2
1.000 0.000 -0.200 0.400 0.000 0.000 0.400 -0.200
0.000 1.000 0.400 0.200 0.000 0.000 0.200 0.400
0.000 0.000 1.600 -1.200 1.000 0.000 -0.200 -0.400
0.000 -0.500 1.000 1.500 0.000 1.000 -0.500 0.000
R4 <- R4 + 0.5*R2
1.000 0.000 -0.200 0.400 0.000 0.000 0.400 -0.200
0.000 1.000 0.400 0.200 0.000 0.000 0.200 0.400
0.000 0.000 1.600 -1.200 1.000 0.000 -0.200 -0.400
0.000 0.000 1.200 1.600 0.000 1.000 -0.400 0.200
R3 <- R3 + 0.75*R4
1.000 0.000 0.000 0.000 0.200 -0.100 0.400 -0.300
0.000 1.000 0.000 0.000 -0.100 -0.200 0.300 0.400
0.000 0.000 1.000 0.000 0.400 0.300 -0.200 -0.100
0.000 0.000 0.000 1.000 -0.300 0.400 -0.100 0.200
Bentuk akhir [I | A^-1], sehingga A^-1 =
0.200 -0.100 0.400 -0.300
-0.100 -0.200 0.300 0.400
0.400 0.300 -0.200 -0.100
-0.300 0.400 -0.100 0.200
Solusi Invers:
0.2 -0.1 0.4 -0.3
-0.1 -0.2 0.3 0.4
0.4 0.3 -0.2 -0.1
-0.3 0.4 -0.1 0.2

```

Gambar 4: Langkah-langkah Eksekusi Kasus Uji 5.2.1 (Metode Augmentasi)

Berikut adalah langkah-langkah dan hasil pencarian invers menggunakan **Metode Adjoin**:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 3

--- Matriks Balikan (Invers) ---
1. Metode Augmentasi
2. Metode Adjoin
Pilih metode (1-2): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/inverse_kasus_5_2_1.txt

Langkah-langkah:
Matriks A:
  0.000  1.000  2.000 -1.000
  1.000  0.000  1.000  2.000
  2.000  1.000  0.000  1.000
 -1.000  2.000  1.000  0.000

Matriks kofaktor C, dengan Cij = (-1)(i+j) × det(Mij):
  4.000 -2.000  8.000 -6.000
 -2.000 -4.000  6.000  8.000
  8.000  6.000 -4.000 -2.000
 -6.000  8.000 -2.000  4.000

det(A) = 20

```

Adjoin(A) = transpos dari matriks kofaktor:

```

  4.000 -2.000  8.000 -6.000
 -2.000 -4.000  6.000  8.000
  8.000  6.000 -4.000 -2.000
 -6.000  8.000 -2.000  4.000

```

$A^{-1} = (1 / 20) \times \text{Adjoin}(A) =$

```

  0.200 -0.100  0.400 -0.300
 -0.100 -0.200  0.300  0.400
  0.400  0.300 -0.200 -0.100
 -0.300  0.400 -0.100  0.200

```

Solusi Invers:

```

0.2 -0.1 0.4 -0.3
-0.1 -0.2 0.3 0.4
0.4 0.3 -0.2 -0.1
-0.3 0.4 -0.1 0.2

```

Gambar 5: Langkah-langkah Eksekusi Kasus Uji 5.2.1 (Metode Adjoin)

Analisis: Keluaran program dari kedua metode terbukti benar, konsisten, dan presisi. Apabila kita mengalikan matriks asal dengan matriks solusi invers tersebut, hasilnya akan kembali menjadi matriks identitas. Selain itu program secara konsisten mempertahankan sifat simetris dari matriks asal, di mana matriks invers yang dihasilkan juga berbentuk simetris atau nilai elemen baris dan kolomnya saling berkebalikan secara cermin.

4.2.2 Kasus Uji 5.2.2

Masukan: Matriks persegi berukuran 5x5.

$$D = \begin{bmatrix} 1 & 2 & 0 & -1 & 3 \\ 2 & -1 & 4 & 0 & 1 \\ 0 & 3 & 1 & 2 & -2 \\ 4 & 0 & 5 & 1 & 2 \\ 3 & 1 & 4 & -1 & 4 \end{bmatrix}$$

Keluaran: Berikut adalah hasil penanganan matriks yang teridentifikasi singular menggunakan Metode Augmentasi dan Adjoin:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 3

--- Matriks Balikan (Invers) ---
1. Metode Augmentasi
2. Metode Adjoin
Pilih metode (1-2): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/inverse_kasus_5_2_2.txt

Langkah-langkah:
det(A) = 0, matriks tidak memiliki invers.
matriks tidak memiliki balikan

```

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 3

--- Matriks Balikan (Invers) ---
1. Metode Augmentasi
2. Metode Adjoin
Pilih metode (1-2): 1
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/inverse_kasus_5_2_2.txt

Langkah-langkah:
det(A) = 0, matriks tidak memiliki invers.
matriks tidak memiliki balikan

```

Gambar 6: Hasil Eksekusi Kasus Uji 5.2.2 (Penanganan Matriks Singular)

Analisis: Penolakan yang dilakukan program sudah sangat tepat. Matriks ini terbukti merupakan matriks singular yang memiliki determinan bernilai nol mutlak akibat adanya kebergantungan linier antar baris penyusunnya. Algoritma program di dalam Metode Augmentasi sukses mendeteksi kegagalan pembentukan pivot utama pada salah satu kolomnya, sementara Metode Adjoin mendeteksinya dari kalkulasi awal determinan. Keduanya langsung menghentikan paksa proses pencarian invers dan memberikan peringatan luaran yang informatif.

4.3 Sistem Persamaan Linier

4.3.1 Kasus Uji 5.3.1

Masukan: Matriks persegi koefisien berukuran 4x4 beserta vektor konstanta berukuran 4x1.

$$A = \begin{bmatrix} 0 & 2 & -1 & 1 \\ 1 & -1 & 2 & 3 \\ 2 & 1 & 1 & -1 \\ -1 & 3 & 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} -5 \\ 15 \\ 1 \\ -3 \end{bmatrix}$$

Keluaran: Berikut adalah penyelesaian matriks koefisien 4x4 menggunakan Metode Matriks Balikan (Invers):

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 3
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/spl_kasus_5_3_1.txt

--- Hasil SPL ---

Langkah-langkah:
Matriks augmentasi [A | I]:
  0.000  2.000 -1.000  1.000  1.000  0.000  0.000  0.000
  1.000 -1.000  2.000  3.000  0.000  1.000  0.000  0.000
  2.000  1.000  1.000 -1.000  0.000  0.000  1.000  0.000
 -1.000  3.000  0.000  2.000  0.000  0.000  0.000  1.000

R1 <-> R3
  2.000  1.000  1.000 -1.000  0.000  0.000  1.000  0.000
  1.000 -1.000  2.000  3.000  0.000  1.000  0.000  0.000
  0.000  2.000 -1.000  1.000  1.000  0.000  0.000  0.000
 -1.000  3.000  0.000  2.000  0.000  0.000  0.000  1.000

R1 <- R1 / 2
  1.000  0.500  0.500 -0.500  0.000  0.000  0.500  0.000
  1.000 -1.000  2.000  3.000  0.000  1.000  0.000  0.000
  0.000  2.000 -1.000  1.000  1.000  0.000  0.000  0.000
 -1.000  3.000  0.000  2.000  0.000  0.000  0.000  1.000

R4 <- R4 / 3.25
  1.000  0.000  0.000 -1.750  0.000 -0.250  0.500 -0.250
  0.000  1.000  0.000  0.083  0.000 -0.083  0.167  0.250
  0.000  0.000  1.000  2.417  0.000  0.583 -0.167  0.250
  0.000  0.000  0.000  1.000  0.308  0.231 -0.154 -0.077

R1 <- R1 + 1.75*R4
  1.000  0.000  0.000  0.000  0.538  0.154  0.231 -0.385
  0.000  1.000  0.000  0.083  0.000 -0.083  0.167  0.250
  0.000  0.000  1.000  2.417  0.000  0.583 -0.167  0.250
  0.000  0.000  0.000  1.000  0.308  0.231 -0.154 -0.077

R2 <- R2 - 0.083*R4
  1.000  0.000  0.000  0.000  0.538  0.154  0.231 -0.385
  0.000  1.000  0.000  0.000 -0.026 -0.103  0.179  0.256
  0.000  0.000  1.000  2.417  0.000  0.583 -0.167  0.250
  0.000  0.000  0.000  1.000  0.308  0.231 -0.154 -0.077

R3 <- R3 - 2.417*R4
  1.000  0.000  0.000  0.000  0.538  0.154  0.231 -0.385
  0.000  1.000  0.000  0.000 -0.026 -0.103  0.179  0.256
  0.000  0.000  1.000  0.000 -0.744  0.026  0.205  0.436
  0.000  0.000  0.000  1.000  0.308  0.231 -0.154 -0.077

R2 <- R2 - 1*R1
  1.000  0.500  0.500 -0.500  0.000  0.000  0.500  0.000
  0.000  3.500  0.500  1.500  0.000  0.000  0.500  1.000
  0.000  2.000 -1.000  1.000  1.000  0.000  0.000  0.000
  0.000 -1.500  1.500  3.500  0.000  1.000 -0.500  0.000

R2 <- R2 / 3.5
  1.000  0.500  0.500 -0.500  0.000  0.000  0.500  0.000
  0.000  1.000  0.143  0.429  0.000  0.000  0.143  0.286
  0.000  2.000 -1.000  1.000  1.000  0.000  0.000  0.000
  0.000 -1.500  1.500  3.500  0.000  1.000 -0.500  0.000

R1 <- R1 - 0.5*R2
  1.000  0.000  0.429 -0.714  0.000  0.000  0.429 -0.143
  0.000  1.000  0.143  0.429  0.000  0.000  0.143  0.286
  0.000  2.000 -1.000  1.000  1.000  0.000  0.000  0.000
  0.000 -1.500  1.500  3.500  0.000  1.000 -0.500  0.000

R3 <- R3 - 2*R2
  1.000  0.000  0.429 -0.714  0.000  0.000  0.429 -0.143
  0.000  1.000  0.143  0.429  0.000  0.000  0.143  0.286
  0.000  0.000 -1.286  0.143  1.000  0.000 -0.286 -0.571
  0.000 -1.500  1.500  3.500  0.000  1.000 -0.500  0.000

Bentuk akhir [I | A^-1], sehingga A^-1 =
  0.538  0.154  0.231 -0.385
 -0.026 -0.103  0.179  0.256
 -0.744  0.026  0.205  0.436
  0.308  0.231 -0.154 -0.077

Matriks konstanta B:
 -5.000
 15.000
 1.000
 -3.000

X = A^-1 * B =
  1.000
 -2.000
 3.000
 2.000

Solusi:
x1 = 1
x2 = -2
x3 = 3
x4 = 2

```

Gambar 7: Hasil Eksekusi Kasus Uji 5.3.1 (Metode Matriks Balikan)

Analisis: Hasil komputasi program sangat akurat dan sesuai dengan hitungan manual matematis. Pada kasus ini, elemen baris pertama dan kolom pertama bernilai nol yang secara alamiah akan menyebabkan galat pembagian dengan nol (*division by zero*) pada algoritma naif. Namun berkat implementasi penukaran baris parsial (*partial pivoting*), program secara otomatis menukar baris pertama dengan baris ketiga yang memiliki nilai mutlak terbesar, sehingga komputasi berjalan lancar hingga akhir tanpa galat.

4.3.2 Kasus Uji 5.3.2

Masukan: Matriks koefisien berukuran 3x5 beserta vektor konstanta berukuran 3x1.

$$A = \begin{bmatrix} 1 & 2 & -1 & 0 & 3 \\ 2 & 4 & 1 & -1 & 5 \\ -1 & -2 & 2 & 3 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 10 \\ -1 \end{bmatrix}$$

Keluaran: Berikut adalah pendeteksian multi solusi dari matriks koefisien berukuran 3x5 menggunakan Metode Eliminasi Gauss:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 1

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 1
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/spl_kasus_5_3_2.txt

--- Hasil SPL ---

Langkah-langkah:
R1 <-> R2
  2.000  4.000  1.000 -1.000  5.000 10.000
  1.000  2.000 -1.000  0.000  3.000  2.000
 -1.000 -2.000  2.000  3.000 -1.000 -1.000
R1 <- R1 / 2
  1.000  2.000  0.500 -0.500  2.500  5.000
  1.000  2.000 -1.000  0.000  3.000  2.000
 -1.000 -2.000  2.000  3.000 -1.000 -1.000

R2 <- R2 - 1*R1
  1.000  2.000  0.500 -0.500  2.500  5.000
  0.000  0.000 -1.500  0.500  0.500 -3.000
 -1.000 -2.000  2.000  3.000 -1.000 -1.000
R3 <- R3 + 1*R1
  1.000  2.000  0.500 -0.500  2.500  5.000
  0.000  0.000 -1.500  0.500  0.500 -3.000
  0.000  0.000  2.500  2.500  1.500  4.000
R2 <-> R3
  1.000  2.000  0.500 -0.500  2.500  5.000
  0.000  0.000  2.500  2.500  1.500  4.000
  0.000  0.000 -1.500  0.500  0.500 -3.000
R2 <- R2 / 2.5
  1.000  2.000  0.500 -0.500  2.500  5.000
  0.000  0.000  1.000  1.000  0.600  1.600
  0.000  0.000 -1.500  0.500  0.500 -3.000
R3 <- R3 + 1.5*R2
  1.000  2.000  0.500 -0.500  2.500  5.000
  0.000  0.000  1.000  1.000  0.600  1.600
  0.000  0.000  0.000  2.000  1.400 -0.600
R3 <- R3 / 2
  1.000  2.000  0.500 -0.500  2.500  5.000
  0.000  0.000  1.000  1.000  0.600  1.600
  0.000  0.000  0.000  1.000  0.700 -0.300

Solusi:
x1 = 3.9 - 2s - 2.9t
x2 = s
x3 = 1.9 + 0.1t
x4 = -0.3 - 0.7t
x5 = t

```

Gambar 8: Hasil Eksekusi Kasus Uji 5.3.2 (Solusi Parametrik)

Analisis: Sistem persamaan ini memiliki jumlah variabel yang lebih banyak daripada jumlah persamaannya, sehingga secara logis tidak mungkin memiliki solusi tunggal. Program secara cerdas mampu mengidentifikasi kolom tanpa pivot pembuka pada bentuk eselon baris, lalu memetakannya menjadi variabel bebas s dan t . Apabila kita menyulihkan kembali formulasi parametrik tersebut ke dalam persamaan asal, seluruh nilainya terbukti konsisten.

4.3.3 Kasus Uji 5.3.3

Masukan: Matriks koefisien berukuran 4×4 beserta vektor konstanta berukuran 4×1 .

$$A = \begin{bmatrix} 1 & -2 & 3 & 1 \\ 2 & 1 & -1 & 4 \\ 0 & 3 & 2 & -2 \\ 3 & -1 & 2 & 5 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ -1 \\ 3 \\ 4 \end{bmatrix}$$

Keluaran: Berikut adalah hasil uji sistem persamaan tak konsisten menggunakan Metode Eliminasi Gauss-Jordan:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 1

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/spl_kasus_5_3_3.txt

--- Hasil SPL ---

Langkah-langkah:
R1 <-> R4
  3.000  -1.000  2.000  5.000  4.000
  2.000   1.000 -1.000  4.000 -1.000
  0.000   3.000  2.000 -2.000  3.000
  1.000 -2.000   3.000  1.000  4.000
R1 <- R1 / 3
  1.000 -0.333  0.667  1.667  1.333
  2.000   1.000 -1.000  4.000 -1.000
  0.000   3.000  2.000 -2.000  3.000
  1.000 -2.000   3.000  1.000  4.000

R2 <- R2 - 2*R1
  1.000 -0.333  0.667  1.667  1.333
  0.000   1.667 -2.333  0.667 -3.667
  0.000   3.000  2.000 -2.000  3.000
  1.000 -2.000   3.000  1.000  4.000
R4 <- R4 - 1*R1
  1.000 -0.333  0.667  1.667  1.333
  0.000   1.667 -2.333  0.667 -3.667
  0.000   3.000  2.000 -2.000  3.000
  0.000 -1.667  2.333 -0.667  2.667
R2 <-> R3
  1.000 -0.333  0.667  1.667  1.333
  0.000   3.000  2.000 -2.000  3.000
  0.000   1.667 -2.333  0.667 -3.667
  0.000 -1.667  2.333 -0.667  2.667
R2 <- R2 / 3
  1.000 -0.333  0.667  1.667  1.333
  0.000   1.000  0.667 -0.667  1.000
  0.000   1.667 -2.333  0.667 -3.667
  0.000 -1.667  2.333 -0.667  2.667
R1 <- R1 + 0.333*R2
  1.000  0.000  0.889  1.444  1.667
  0.000   1.000  0.667 -0.667  1.000
  0.000   1.667 -2.333  0.667 -3.667
  0.000 -1.667  2.333 -0.667  2.667
R3 <- R3 - 1.667*R2
  1.000  0.000  0.889  1.444  1.667
  0.000   1.000  0.667 -0.667  1.000
  0.000   0.000 -3.444  1.778 -5.333
  0.000 -1.667  2.333 -0.667  2.667
R4 <- R4 + 1.667*R2
  1.000  0.000  0.889  1.444  1.667
  0.000   1.000  0.667 -0.667  1.000
  0.000   0.000 -3.444  1.778 -5.333
  0.000  0.000  3.444 -1.778  4.333

R3 <-> R4
  1.000  0.000  0.889  1.444  1.667
  0.000   1.000  0.667 -0.667  1.000
  0.000   0.000  3.444 -1.778  4.333
  0.000   0.000 -3.444  1.778 -5.333
R3 <- R3 / 3.444
  1.000  0.000  0.889  1.444  1.667
  0.000   1.000  0.667 -0.667  1.000
  0.000   0.000  1.000 -0.516  1.258
  0.000   0.000 -3.444  1.778 -5.333
R1 <- R1 - 0.889*R3
  1.000  0.000  0.000  1.903  0.548
  0.000   1.000  0.667 -0.667  1.000
  0.000   0.000  1.000 -0.516  1.258
  0.000   0.000 -3.444  1.778 -5.333
R2 <- R2 - 0.667*R3
  1.000  0.000  0.000  1.903  0.548
  0.000   1.000  0.000 -0.323  0.161
  0.000   0.000  1.000 -0.516  1.258
  0.000   0.000 -3.444  1.778 -5.333
R4 <- R4 + 3.444*R3
  1.000  0.000  0.000  1.903  0.548
  0.000   1.000  0.000 -0.323  0.161
  0.000   0.000  1.000 -0.516  1.258
  0.000   0.000  0.000  0.000 -1.000

Solusi:
solusi tidak ada

```

Gambar 9: Hasil Eksekusi Kasus Uji 5.3.3 (Sistem Tak Konsisten)

Analisis: Penolakan yang dilakukan oleh program sudah sangat tepat. Apabila diamati secara teliti, baris keempat pada matriks koefisien tersebut merupakan hasil penjumlahan langsung dari baris pertama dan baris kedua. Kebergantungan linier ini menyebabkan sistem ini terbukti tidak konsisten dan program secara tepat mengidentifikasi baris kontradiksi sehingga titik potong solusi dinyatakan tidak ada.

4.3.4 Kasus Uji 5.3.4

Masukan: Matriks diskrit Laplacian satu dimensi berukuran 6×6 dan 10×10 .

Keluaran: Berikut adalah solusi untuk matriks diskrit Laplacian menggunakan Kaidah Cramer:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 1

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 4
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 2
Masukkan path/nama file: test/spl_kasus_5_3_4_n6.txt

--- Hasil SPL ---

Langkah-langkah:
Matriks koefisien A:
  2.000  -1.000  0.000  0.000  0.000  0.000
 -1.000  2.000 -1.000  0.000  0.000  0.000
  0.000 -1.000  2.000 -1.000  0.000  0.000
  0.000  0.000 -1.000  2.000 -1.000  0.000
  0.000  0.000  0.000 -1.000  2.000 -1.000
  0.000  0.000  0.000  0.000 -1.000  2.000
det(A) = 7

```

```

A1 (kolom 1 diganti konstanta):
  1.000  -1.000  0.000  0.000  0.000  0.000
  1.000  2.000 -1.000  0.000  0.000  0.000
  1.000 -1.000  2.000 -1.000  0.000  0.000
  1.000  0.000 -1.000  2.000 -1.000  0.000
  1.000  0.000  0.000 -1.000  2.000 -1.000
  1.000  0.000  0.000  0.000 -1.000  2.000
det(A1) = 21
x1 = det(A1) / det(A) = 21 / 7 = 3

A2 (kolom 2 diganti konstanta):
  2.000  1.000  0.000  0.000  0.000  0.000
 -1.000  1.000 -1.000  0.000  0.000  0.000
  0.000  1.000  2.000 -1.000  0.000  0.000
  0.000  1.000 -1.000  2.000 -1.000  0.000
  0.000  1.000  0.000 -1.000  2.000 -1.000
  0.000  1.000  0.000  0.000 -1.000  2.000
det(A2) = 35
x2 = det(A2) / det(A) = 35 / 7 = 5

A3 (kolom 3 diganti konstanta):
  2.000 -1.000  1.000  0.000  0.000  0.000
 -1.000  2.000  1.000  0.000  0.000  0.000
  0.000 -1.000  1.000 -1.000  0.000  0.000
  0.000  0.000  1.000  2.000 -1.000  0.000
  0.000  0.000  1.000 -1.000  2.000 -1.000
  0.000  0.000  1.000  0.000 -1.000  2.000
det(A3) = 42
x3 = det(A3) / det(A) = 42 / 7 = 6

A4 (kolom 4 diganti konstanta):
  2.000 -1.000  0.000  1.000  0.000  0.000
 -1.000  2.000 -1.000  1.000  0.000  0.000
  0.000 -1.000  2.000  1.000  0.000  0.000
  0.000  0.000 -1.000  1.000 -1.000  0.000
  0.000  0.000  0.000  1.000  2.000 -1.000
  0.000  0.000  0.000  1.000 -1.000  2.000
det(A4) = 42
x4 = det(A4) / det(A) = 42 / 7 = 6

```

```

A5 (kolom 5 diganti konstanta):
  2.000 -1.000  0.000  0.000  1.000  0.000
 -1.000  2.000 -1.000  0.000  1.000  0.000
  0.000 -1.000  2.000 -1.000  1.000  0.000
  0.000  0.000 -1.000  2.000  1.000  0.000
  0.000  0.000  0.000 -1.000  1.000 -1.000
  0.000  0.000  0.000  0.000  1.000  2.000
det(A5) = 35
x5 = det(A5) / det(A) = 35 / 7 = 5

A6 (kolom 6 diganti konstanta):
  2.000 -1.000  0.000  0.000  0.000  1.000
 -1.000  2.000 -1.000  0.000  0.000  1.000
  0.000 -1.000  2.000 -1.000  0.000  1.000
  0.000  0.000 -1.000  2.000 -1.000  1.000
  0.000  0.000  0.000 -1.000  2.000  1.000
  0.000  0.000  0.000  0.000 -1.000  1.000
det(A6) = 21
x6 = det(A6) / det(A) = 21 / 7 = 3

Solusi:
x1 = 3
x2 = 5
x3 = 6
x4 = 6
x5 = 5
x6 = 3

```

Gambar 10: Hasil Eksekusi Kasus Uji 5.3.4 (Matriks Laplacian Diskrit)

Analisis: Solusi eksak sistem ini mengikuti rumusan matematis $x_i = \frac{i(n+1-i)}{2}$ yang merupakan bentuk diskrit dari fungsi parabola. Hasil komputasi program terbukti sama persis dengan rumusan tersebut. Matriks Laplacian memiliki sifat dominan diagonal,

sehingga komputasi berjalan sangat stabil tanpa menimbulkan galat pembulatan yang terlihat.

4.4 SPL Berbentuk Matriks Augmented

4.4.1 Kasus Uji 5.4.1

Masukan: Sistem persamaan linier berformat matriks augmented 4x5.

$$[A|b] = \begin{bmatrix} 0 & 1 & -1 & 2 & -1 \\ 2 & 0 & 1 & -1 & 3 \\ -1 & 2 & 0 & 1 & -3 \\ 1 & -1 & 2 & 0 & 9 \end{bmatrix}$$

Keluaran: Berikut adalah hasil pencarian solusi menggunakan keempat metode yang diimplementasikan pada program:

1. Metode Eliminasi Gauss

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 1
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 4
Jumlah kolom: 5
Masukkan isi matriks:
Masukkan baris 1: 0 1 -1 2 -1
Masukkan baris 2: 2 0 1 -1 3
Masukkan baris 3: -1 2 0 1 -3
Masukkan baris 4: 1 -1 2 0 9

--- Hasil SPL ---

Langkah-langkah:
R1 <-> R2
  2.000    0.000    1.000   -1.000    3.000
  0.000    1.000   -1.000    2.000   -1.000
 -1.000    2.000    0.000    1.000   -3.000
  1.000   -1.000    2.000    0.000    9.000

R3 <- R3 - 1*R2
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    0.000   -1.250    1.750   -0.250
  0.000   -1.000    1.500    0.500    7.500

R4 <- R4 + 1*R2
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    0.000   -1.250    1.750   -0.250
  0.000    0.000    1.750    0.750    6.750

R3 <-> R4
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    0.000    1.750    0.750    6.750
  0.000    0.000   -1.250    1.750   -0.250

R3 <- R3 / 1.75
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    0.000    1.000    0.429    3.857
  0.000    0.000   -1.250    1.750   -0.250

R4 <- R4 + 1.25*R3
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    0.000    1.000    0.429    3.857
  0.000    0.000    0.000    2.286    4.571

R1 <- R1 / 2
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000   -1.000    2.000   -1.000
 -1.000    2.000    0.000    1.000   -3.000
  1.000   -1.000    2.000    0.000    9.000

R3 <- R3 + 1*R1
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000   -1.000    2.000   -1.000
  0.000    2.000    0.500    0.500   -1.500
  1.000   -1.000    2.000    0.000    9.000

R4 <- R4 - 1*R1
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000   -1.000    2.000   -1.000
  0.000    2.000    0.500    0.500   -1.500
  0.000   -1.000    1.500    0.500    7.500

R2 <-> R3
  1.000    0.000    0.500   -0.500    1.500
  0.000    2.000    0.500    0.500   -1.500
  0.000    1.000   -1.000    2.000   -1.000
  0.000   -1.000    1.500    0.500    7.500

R2 <- R2 / 2
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    1.000   -1.000    2.000   -1.000
  0.000   -1.000    1.500    0.500    7.500

R4 <- R4 / 2.286
  1.000    0.000    0.500   -0.500    1.500
  0.000    1.000    0.250    0.250   -0.750
  0.000    0.000    1.000    0.429    3.857
  0.000    0.000    0.000    1.000    2.000

Solusi:
x1 = 1
x2 = -2
x3 = 3
x4 = 2

```

Gambar 11: Langkah-langkah Eksekusi Kasus Uji 5.4.1 (Metode Gauss)

2. Metode Eliminasi Gauss-Jordan

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 3
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 0 1 -1 2 -1
Input harus berupa bilangan bulat.
Jumlah baris: 4
Jumlah kolom: 5
Masukkan isi matriks:
Masukkan baris 1: 0 1 -1 2 -1
Masukkan baris 2: 2 0 1 -1 3
Masukkan baris 3: -1 2 0 1 -3
Masukkan baris 4: 1 -1 2 0 9

--- Hasil SPL ---

Langkah-langkah:
Matriks augmentasi [A | I]:
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
2.000 0.000 1.000 -1.000 0.000 1.000 0.000 0.000
-1.000 2.000 0.000 1.000 0.000 0.000 1.000 0.000
1.000 -1.000 2.000 0.000 0.000 0.000 0.000 1.000

R2 <- R2 / 2
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 0.250 0.250 0.000 0.250 0.500 0.000
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
0.000 -1.000 1.500 0.500 0.000 -0.500 0.000 1.000

R3 <- R3 - 1*R2
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 0.250 0.250 0.000 0.250 0.500 0.000
0.000 0.000 -1.250 1.750 1.000 -0.250 -0.500 0.000
0.000 -1.000 1.500 0.500 0.000 -0.500 0.000 1.000

R4 <- R4 + 1*R2
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 0.250 0.250 0.000 0.250 0.500 0.000
0.000 0.000 -1.250 1.750 1.000 -0.250 -0.500 0.000
0.000 0.000 1.750 0.750 0.000 -0.250 0.500 1.000

R3 <-> R4
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 0.250 0.250 0.000 0.250 0.500 0.000
0.000 0.000 1.750 0.750 0.000 -0.250 0.500 1.000
0.000 0.000 -1.250 1.750 1.000 -0.250 -0.500 0.000

R3 <- R3 / 1.75
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 0.250 0.250 0.000 0.250 0.500 0.000
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 -1.250 1.750 1.000 -0.250 -0.500 0.000

R2 <- R2 - 0.143*R4
1.000 0.000 0.000 0.000 0.313 0.438 -0.188 -0.063
0.000 1.000 0.000 0.000 -0.063 0.313 0.438 -0.187
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 0.000 1.000 0.438 -0.188 -0.063 0.312

R3 <- R3 - 0.429*R4
1.000 0.000 0.000 0.000 0.313 0.438 -0.188 -0.063
0.000 1.000 0.000 0.000 -0.063 0.313 0.438 -0.187
0.000 0.000 1.000 0.000 -0.188 -0.063 0.313 0.438
0.000 0.000 0.000 1.000 0.438 -0.188 -0.063 0.312

Bentuk akhir [I | A^-1], sehingga A^-1 =
0.313 0.438 -0.188 -0.063
-0.063 0.313 0.438 -0.187
-0.188 -0.063 0.313 0.438
0.438 -0.188 -0.063 0.312

R1 <-> R2
2.000 0.000 1.000 -1.000 0.000 1.000 0.000 0.000
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
-1.000 2.000 0.000 1.000 0.000 0.000 1.000 0.000
1.000 -1.000 2.000 0.000 0.000 0.000 0.000 1.000

R1 <- R1 / 2
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
-1.000 2.000 0.000 1.000 0.000 0.000 1.000 0.000
1.000 -1.000 2.000 0.000 0.000 0.000 0.000 1.000

R3 <- R3 + 1*R1
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
0.000 2.000 0.500 0.500 0.000 0.500 1.000 0.000
1.000 -1.000 2.000 0.000 0.000 0.000 0.000 1.000

R4 <- R4 - 1*R1
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
0.000 2.000 0.500 0.500 0.000 0.500 1.000 0.000
0.000 -1.000 1.500 0.500 0.000 -0.500 0.000 1.000

R2 <-> R3
1.000 0.000 0.500 -0.500 0.000 0.500 0.000 0.000
0.000 2.000 0.500 0.500 0.000 0.500 1.000 0.000
0.000 1.000 -1.000 2.000 1.000 0.000 0.000 0.000
0.000 -1.000 1.500 0.500 0.000 -0.500 0.000 1.000

R1 <- R1 - 0.5*R3
1.000 0.000 0.000 -0.714 0.000 0.571 -0.143 -0.286
0.000 1.000 0.250 0.250 0.000 0.250 0.500 0.000
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 -1.250 1.750 1.000 -0.250 -0.500 0.000

R2 <- R2 - 0.25*R3
1.000 0.000 0.000 -0.714 0.000 0.571 -0.143 -0.286
0.000 1.000 0.000 0.143 0.000 0.286 0.429 -0.143
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 -1.250 1.750 1.000 -0.250 -0.500 0.000

R4 <- R4 + 1.25*R3
1.000 0.000 0.000 -0.714 0.000 0.571 -0.143 -0.286
0.000 1.000 0.000 0.143 0.000 0.286 0.429 -0.143
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 0.000 2.286 1.000 -0.429 -0.143 0.714

R4 <- R4 / 2.286
1.000 0.000 0.000 -0.714 0.000 0.571 -0.143 -0.286
0.000 1.000 0.000 0.143 0.000 0.286 0.429 -0.143
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 0.000 1.000 0.438 -0.188 -0.063 0.312

R1 <- R1 + 0.714*R4
1.000 0.000 0.000 0.000 0.313 0.438 -0.188 -0.063
0.000 1.000 0.000 0.143 0.000 0.286 0.429 -0.143
0.000 0.000 1.000 0.429 0.000 -0.143 0.286 0.571
0.000 0.000 0.000 1.000 0.438 -0.188 -0.063 0.312

X = A^-1 * B =
1.000
-2.000
3.000
2.000

Solusi:
x1 = 1
x2 = -2
x3 = 3
x4 = 2

```

Gambar 12: Langkah-langkah Eksekusi Kasus Uji 5.4.1 (Metode Gauss-Jordan)

Analisis Kasus 5.4.1: Kasus ini menguji ketahanan modul perhitungan sistem persamaan linier terhadap bentuk matriks *augmented* langsung. Matriks dengan dimensi 4×5 ini memiliki nilai koefisien yang saling menyilang. Hasil luaran program untuk seluruh metode (termasuk Invers dan Cramer) terbukti menghasilkan titik potong solusi tunggal yang persis sama, yakni $x_1 = 1, x_2 = -2, x_3 = 3$, dan $x_4 = 2$. Substitusi nilai variabel kembali ke dalam persamaan asal membuktikan bahwa hasil komputasi rentetan OBE oleh program telah sepenuhnya valid tanpa galat logika.

4.4.2 Kasus Uji 5.4.2

Masukan: Sistem persamaan linier berformat matriks augmented 4x7.

$$[A|b] = \begin{bmatrix} 1 & 0 & 2 & -1 & 0 & 3 & -7 \\ 0 & 1 & -1 & 2 & 1 & 0 & 6 \\ 2 & -1 & 0 & 1 & 3 & -2 & 13 \\ 1 & 1 & 1 & 1 & 1 & 1 & 3 \end{bmatrix}$$

Keluaran: Berikut adalah luaran matriks augmented 4x7 yang menghasilkan solusi parametrik menggunakan **Metode Eliminasi Gauss-Jordan**:

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 4
Jumlah kolom: 7
Masukkan isi matriks:
Masukkan baris 1: 1 0 2 -1 0 3 -7
Masukkan baris 2: 0 1 -1 2 1 0 6
Masukkan baris 3: 2 -1 0 1 3 -2 13
Masukkan baris 4: 1 1 1 1 1 1 3

--- Hasil SPL ---

Langkah-langkah:
R1 <-> R3
  2.000  -1.000  0.000  1.000  3.000  -2.000  13.000
  0.000  1.000  -1.000  2.000  1.000  0.000  6.000
  1.000  0.000  2.000  -1.000  0.000  3.000  -7.000
  1.000  1.000  1.000  1.000  1.000  1.000  3.000

R1 <- R1 + 0.5*R2
  1.000  0.000  0.333  0.667  1.333  -0.333  5.333
  0.000  1.000  -0.667  0.333  -0.333  1.333  -2.333
  0.000  0.500  2.000  -1.500  4.000  -13.500
  0.000  1.000  -1.000  2.000  1.000  0.000  6.000

R3 <- R3 - 0.5*R2
  1.000  0.000  0.333  0.667  1.333  -0.333  5.333
  0.000  1.000  -0.667  0.333  -0.333  1.333  -2.333
  0.000  0.000  1.667  -1.667  -1.333  3.333  -12.333
  0.000  1.000  -1.000  2.000  1.000  0.000  6.000

R4 <- R4 - 1*R2
  1.000  0.000  0.333  0.667  1.333  -0.333  5.333
  0.000  1.000  -0.667  0.333  -0.333  1.333  -2.333
  0.000  0.000  1.667  -1.667  -1.333  3.333  -12.333
  0.000  0.000  -1.667  1.667  1.333  -1.333  8.333

R3 <- R3 / 1.667
  1.000  0.000  0.333  0.667  1.333  -0.333  5.333
  0.000  1.000  -0.667  0.333  -0.333  1.333  -2.333
  0.000  0.000  1.000  -1.000  -0.800  2.000  -7.400
  0.000  0.000  -1.667  1.667  1.333  -1.333  8.333

R1 <- R1 - 0.333*R3
  1.000  0.000  0.000  1.000  1.600  -1.000  7.800
  0.000  1.000  -0.667  0.333  -0.333  1.333  -2.333
  0.000  0.000  1.000  -1.000  -0.800  2.000  -7.400
  0.000  0.000  -1.667  1.667  1.333  -1.333  8.333

R2 <- R2 - 0.667*R3
  1.000  0.000  0.000  1.000  1.600  -1.000  7.800
  0.000  1.000  0.000  1.000  0.200  0.000  2.600
  0.000  0.000  1.000  -1.000  -0.800  2.000  -7.400
  0.000  0.000  -1.667  1.667  1.333  -1.333  8.333

R4 <- R4 + 1.667*R3
  1.000  0.000  0.000  1.000  1.600  -1.000  7.800
  0.000  1.000  0.000  1.000  0.200  0.000  2.600
  0.000  0.000  1.000  -1.000  -0.800  2.000  -7.400
  0.000  0.000  0.000  0.000  0.000  2.000  -4.000

R4 <- R4 / 2
  1.000  0.000  0.000  1.000  1.600  -1.000  7.800
  0.000  1.000  0.000  1.000  0.200  0.000  2.600
  0.000  0.000  1.000  -1.000  -0.800  2.000  -7.400
  0.000  0.000  0.000  0.000  0.000  1.000  -2.000

R1 <- R1 + 1*R4
  1.000  0.000  0.000  1.000  1.600  0.000  5.800
  0.000  1.000  0.000  1.000  0.200  0.000  2.600
  0.000  0.000  1.000  -1.000  -0.800  2.000  -7.400
  0.000  0.000  0.000  0.000  0.000  1.000  -2.000

R3 <- R3 - 2*R4
  1.000  0.000  0.000  1.000  1.600  0.000  5.800
  0.000  1.000  0.000  1.000  0.200  0.000  2.600
  0.000  0.000  1.000  -1.000  -0.800  0.000  -3.400
  0.000  0.000  0.000  0.000  0.000  1.000  -2.000

Solusi:
x1 = 5.8 - 1s - 1.6t
x2 = 2.6 - s - 0.2t
x3 = -3.4 + 1s + 0.8t
x4 = s
x5 = t
x6 = -2

```

Gambar 13: Langkah-langkah Eksekusi Kasus Uji 5.4.2 (Augmented Parametrik)

Berikut adalah respon penolakan komputasi untuk dimensi matriks yang tidak didukung oleh **Kaidah Cramer**:

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 4
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 4
Jumlah kolom: 7
Masukkan isi matriks:
Masukkan baris 1: 1 0 2 -1 0 3 -7
Masukkan baris 2: 0 1 -1 2 1 0 6
Masukkan baris 3: 2 -1 0 1 3 -2 13
Masukkan baris 4: 1 1 1 1 1 1 3

--- Hasil SPL ---

Langkah-langkah:
Kaidah Cramer hanya berlaku untuk SPL dengan jumlah persamaan sama dengan jumlah variabel.

Solusi:
solusi tidak ada

```

Gambar 14: Penolakan Kasus Uji 5.4.2 oleh Metode Kaidah Cramer

Analisis Kasus 5.4.2: Sistem dengan empat persamaan dan enam variabel linier ini secara matematis dipastikan tidak akan memiliki solusi titik tunggal karena jumlah variabel yang tidak diketahui jauh melampaui jumlah batas persamaannya. Pada metode komputasi analitik murni seperti Kaidah Cramer, program memunculkan peringatan penolakan otomatis karena matriks koefisien tidak berwujud persegi. Sebaliknya, metode eliminasi Gauss-Jordan sukses menyisakan variabel bebas yang kemudian secara algoritmik dipetakan oleh program menjadi parameter asimtotik s dan t . Menariknya, variabel x_6 tetap terkunci pada nilai tunggal negatif 2 ($x_6 = -2$) terlepas dari berapapun nilai masukan parameter lainnya. Modul substitusi mundur parametrik program sukses memfasilitasi kondisi unik ini tanpa mengalami kebingungan pemetaan variabel.

4.5 SPL Berbentuk Umum

4.5.1 Kasus Uji 5.5.1

Masukan: Empat persamaan linier dengan empat variabel yang disederhanakan menjadi matriks koefisien augmented 4×5 .

$$[A|b] = \begin{bmatrix} 3 & 2 & -1 & 4 & 5 \\ 1 & -3 & 2 & 1 & -2 \\ 4 & 1 & 5 & -2 & 7 \\ 2 & -1 & 3 & 1 & 3 \end{bmatrix}$$

Keluaran: Berikut adalah penyelesaian persamaan linier yang disederhanakan menggunakan **Kaidah Cramer**:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 1

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 4
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 4
Jumlah kolom: 5
Masukkan isi matriks:
Masukkan baris 1: 3 2 -1 4 5
Masukkan baris 2: 1 -3 2 1 -2
Masukkan baris 3: 4 1 5 -2 7
Masukkan baris 4: 2 -1 3 1 3

--- Hasil SPL ---

Langkah-langkah:
Matriks koefisien A:
    3.000    2.000   -1.000    4.000
    1.000   -3.000    2.000    1.000
    4.000    1.000    5.000   -2.000
    2.000   -1.000    3.000    1.000
det(A) = -88

A1 (kolom 1 diganti konstanta):
    5.000    2.000   -1.000    4.000
   -2.000   -3.000    2.000    1.000
    7.000    1.000    5.000   -2.000
    3.000   -1.000    3.000    1.000
det(A1) = 10
x1 = det(A1) / det(A) = 10 / -88 = -0.114

A2 (kolom 2 diganti konstanta):
    3.000    5.000   -1.000    4.000
    1.000   -2.000    2.000    1.000
    4.000    7.000    5.000   -2.000
    2.000    3.000    3.000    1.000
det(A2) = -162
x2 = det(A2) / det(A) = -162 / -88 = 1.841

A3 (kolom 3 diganti konstanta):
    3.000    2.000    5.000    4.000
    1.000   -3.000   -2.000    1.000
    4.000    1.000    7.000   -2.000
    2.000   -1.000    3.000    1.000
det(A3) = -126
x3 = det(A3) / det(A) = -126 / -88 = 1.432

A4 (kolom 4 diganti konstanta):
    3.000    2.000   -1.000    5.000
    1.000   -3.000    2.000   -2.000
    4.000    1.000    5.000    7.000
    2.000   -1.000    3.000    3.000
det(A4) = -68
x4 = det(A4) / det(A) = -68 / -88 = 0.773

Solusi:
x1 = -0.114
x2 = 1.841
x3 = 1.432
x4 = 0.773

```

Gambar 15: Hasil Eksekusi Kasus Uji 5.5.1 (SPL Umum menggunakan Kaidah Cramer)

Analisis: Kehadiran nilai determinan utama bernilai negatif 88 ($\det(A) = -88$) sebagai angka pembagi pada komputasi menghasilkan nilai variabel berupa deret pecahan desimal yang panjang. Program secara otomatis memotong dan membulatkannya menjadi tiga angka di belakang koma (misalnya $x_1 = -0.114$) untuk menjaga kerapian antarmuka pengguna tanpa merusak keutuhan nilai numerik kalkulasi di latar belakang.

4.5.2 Kasus Uji 5.5.2

Masukan: Sembilan persamaan saling silang dengan sembilan variabel pembentuk.

Keluaran: Berikut adalah hasil eksekusi sembilan persamaan saling silang menggunakan Metode Eliminasi Gauss-Jordan:

```

Metode: Eliminasi Gauss-Jordan

Matriks Input:
1 1 1 0 0 0 0 0 0 10
0 0 0 1 1 1 0 0 0 12
0 0 0 0 0 0 1 1 1 9
0.035 0.68 0 0.035 0 0.68 0.035 0 0.554 8.42
0.877 0 0.3 0.877 0.3 0 0.877 0.3 0 11.05
0.035 0.554 0.68 0.035 0.68 0 0.035 0 0 6.19
1 0 0 0 1 0 0 0 1 11
0 1 0 1 0 0 0 1 0 8.5
0 0 1 0 0 1 1 0 0 11.5

Hasil:
x1 = 5.781 - 0.462s
x2 = 4.399 - 0.387s
x3 = -0.18 + 0.849s
x4 = -0.725 + 0.971s
x5 = 5.219 - 0.538s
x6 = 7.506 - 0.433s
x7 = 4.174 - 0.416s
x8 = 4.826 - 0.584s
x9 = s

```

Gambar 16: Hasil Eksekusi Kasus Uji 5.5.2 (SPL Umum Multi-Solusi Parametrik)

Analisis: Walaupun jumlah persamaan dan jumlah variabel terlihat seimbang (sembilan persamaan), gabungan relasi antar baris pembentuk matriks ini memiliki total ruas kanan yang bergantung linier secara ekuivalen. Kebergantungan linier ini menyebabkan matriks kehilangan status *full rank*, sehingga determinannya menjadi nol dan persamaan tidak memiliki titik potong variabel tunggal mutlak. Program mendeteksi redundansi tersebut lalu secara cerdas memprosesnya menggunakan pemetaan parametrik, menjadikan x_9 sebagai parameter asimtotik lepas s . Semua nilai sisa variabel lain dipetakan dan dijawab ke dalam rentang ekuivalensi terhadap nilai bebas parameter s tersebut.

4.6 Aplikasi Sistem Persamaan Linier

4.6.1 Kasus Uji 5.6

Masukan: Diberikan dataset kecil yang berisi informasi luas rumah (X) dan harga rumah (Y):

X (luas rumah)	Y (harga)
1	3
2	5
3	6

Model prediksi linier dibentuk dalam bentuk $y = \theta_0 + \theta_1 x$. Berdasarkan persamaan normal regresi $\theta = (X^T X)^{-1} X^T Y$, terbentuk sistem persamaan linier dengan matriks koefisien dan konstanta:

$$\begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} 14 \\ 31 \end{bmatrix}$$

Keluaran: Berikut adalah luaran penyelesaian parameter θ_0 dan θ_1 menggunakan berbagai metode pada program:

1. Metode Eliminasi Gauss dan Gauss-Jordan

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 1
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 2
Jumlah kolom: 3
Masukkan isi matriks:
Masukkan baris 1: 3 6 14
Masukkan baris 2: 6 14 31

--- Hasil SPL ---

Langkah-langkah:
R1 <-> R2
    6.000    14.000    31.000
    3.000     6.000    14.000
R1 <- R1 / 6
    1.000     2.333     5.167
    3.000     6.000    14.000
R2 <- R2 - 3*R1
    1.000     2.333     5.167
    0.000    -1.000    -1.500
R2 <- R2 / -1
    1.000     2.333     5.167
    0.000     1.000     1.500

Solusi:
x1 = 1.667
x2 = 1.5

```

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 2
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 2
Jumlah kolom: 3
Masukkan isi matriks:
Masukkan baris 1: 3 6 14
Masukkan baris 2: 6 14 31

--- Hasil SPL ---

Langkah-langkah:
R1 <-> R2
    6.000    14.000    31.000
    3.000     6.000    14.000
R1 <- R1 / 6
    1.000     2.333     5.167
    3.000     6.000    14.000
R2 <- R2 - 3*R1
    1.000     2.333     5.167
    0.000    -1.000    -1.500
R2 <- R2 / -1
    1.000     2.333     5.167
    0.000     1.000     1.500
R1 <- R1 - 2.333*R2
    1.000     0.000     1.667
    0.000     1.000     1.500

Solusi:
x1 = 1.667
x2 = 1.5

```

Gambar 17: Hasil Eksekusi Kasus Uji 5.6 (Eliminasi Gauss dan Gauss-Jordan)

2. Metode Matriks Balikan (Invers) dan Kaidah Cramer

```

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 3
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 2
Jumlah kolom: 3
Masukkan isi matriks:
Masukkan baris 1: 3 6 14
Masukkan baris 2: 6 14 31

--- Hasil SPL ---

Langkah-langkah:
Matriks augmentasi [A | I]:
  3.000  6.000  1.000  0.000
  6.000 14.000  0.000  1.000
R1 <-> R2
  6.000 14.000  0.000  1.000
  3.000  6.000  1.000  0.000
R1 <- R1 / 6
  1.000  2.333  0.000  0.167
  3.000  6.000  1.000  0.000
R2 <- R2 - 3*R1
  1.000  2.333  0.000  0.167
  0.000 -1.000  1.000 -0.500
R2 <- R2 / -1
  1.000  2.333  0.000  0.167
  0.000  1.000 -1.000  0.500
R1 <- R1 - 2.333*R2
  1.000  0.000  2.333 -1.000
  0.000  1.000 -1.000  0.500

Bentuk akhir [I | A^-1], sehingga A^-1 =
  2.333 -1.000
 -1.000  0.500

Matriks konstanta B:
 14.000
 31.000

X = A^-1 * B =
 1.667
 1.500

Solusi:
x1 = 1.667
x2 = 1.5

--- Sistem Persamaan Linier (SPL) ---
1. Metode Eliminasi Gauss
2. Metode Eliminasi Gauss-Jordan
3. Metode Matriks Balikan
4. Kaidah Cramer
Pilih metode (1-4): 4
Sumber input:
1. Manual (Keyboard)
2. File (.txt)
Pilih (1-2): 1
Jumlah baris: 2
Jumlah kolom: 3
Masukkan isi matriks:
Masukkan baris 1: 3 6 14
Masukkan baris 2: 6 14 31

--- Hasil SPL ---

Langkah-langkah:
Matriks koefisien A:
  3.000  6.000
  6.000 14.000
det(A) = 6

A1 (kolom 1 diganti konstanta):
 14.000  6.000
 31.000 14.000
det(A1) = 10
x1 = det(A1) / det(A) = 10 / 6 = 1.667

A2 (kolom 2 diganti konstanta):
  3.000 14.000
  6.000 31.000
det(A2) = 9
x2 = det(A2) / det(A) = 9 / 6 = 1.5

Solusi:
x1 = 1.667
x2 = 1.5

```

Gambar 18: Hasil Eksekusi Kasus Uji 5.6 (Matriks Balikan dan Kaidah Cramer)

Analisis: Hasil komputasi dari keempat metode menunjukkan konsistensi penuh

dalam menentukan parameter model, di mana nilai $\theta_0 = 1.667$ dan $\theta_1 = 1.5$. Hal ini membuktikan bahwa perumusan model prediksi harga rumah terhadap luas bangunan melalui persamaan normal regresi dapat diselesaikan secara presisi menggunakan berbagai algoritma sistem persamaan linier.

4.7 Interpolasi Polinomial

4.7.1 Kasus Uji 5.7 Studi Kasus 1

Berikut adalah hasil penaksiran interpolasi dari tujuh pasang titik data turunan fungsi logaritmik:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 4

--- Interpolasi Polinomial ---
Masukkan jumlah titik (N): 7
Masukkan titik-titik (x y):
Masukkan baris 1: 0.2 0.182
Masukkan baris 2: 0.4 0.336
Masukkan baris 3: 0.6 0.470
Masukkan baris 4: 0.8 0.588
Masukkan baris 5: 1.0 0.693
Masukkan baris 6: 1.2 0.788
Masukkan baris 7: 1.4 0.875
Masukkan nilai x yang ditaksir: 0.3

Persamaan Interpolasi:
P(x) = 1.003x - 0.546x^2 + 0.479x^3 - 0.408x^4 + 0.208x^5 - 0.043x^6
Nilai p(0.3) = 0.262

Persamaan Interpolasi:
P(x) = 1.003x - 0.546x^2 + 0.479x^3 - 0.408x^4 + 0.208x^5 - 0.043x^6
Nilai p(0.95) = 0.668

Persamaan Interpolasi:
P(x) = 1.003x - 0.546x^2 + 0.479x^3 - 0.408x^4 + 0.208x^5 - 0.043x^6
Nilai p(0.55) = 0.438

Persamaan Interpolasi:
P(x) = 1.003x - 0.546x^2 + 0.479x^3 - 0.408x^4 + 0.208x^5 - 0.043x^6
Nilai p(1.38) = 0.867

```

Gambar 19: Hasil Eksekusi Kasus Uji 5.7 - Studi Kasus 1 (Interpolasi Logaritmik)

Analisis: Fungsi polinomial yang terbentuk sangat akurat dalam memodelkan titik data masukan. Nilai koefisien konstanta bernilai nol mutlak yang sangat masuk akal mengingat data masukan tersebut berawal dari absis 0.2. Hasil evaluasi pada berbagai titik uji juga tercatat sangat presisi dan ekuivalen dengan perhitungan fungsi logaritma aslinya, di mana deviasi yang muncul murni bersumber dari pembulatan data masukan awal, bukan dari kelemahan metode.

4.7.2 Kasus Uji 5.7 Studi Kasus 2

Berikut adalah hasil interpolasi eksponensial suhu pemanas terhadap waktu:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 4

--- Interpolasi Polinomial ---
Masukkan jumlah titik (N): 6
Masukkan titik-titik (x y):
Masukkan baris 1: 0 25.0
Masukkan baris 2: 2 31.2
Masukkan baris 3: 4 38.6
Masukkan baris 4: 6 47.5
Masukkan baris 5: 8 58.1
Masukkan baris 6: 10 70.8
Masukkan nilai x yang ditaksir: 1

Persamaan Interpolasi:
P(x) = 25.000 + 2.892x + 0.070x^2 + 0.020x^3 - 0.002x^4 + 0.000x^5
Nilai p(1) = 27.981

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 4

--- Interpolasi Polinomial ---
Masukkan jumlah titik (N): 6
Masukkan titik-titik (x y):
Masukkan baris 1: 0 25.0
Masukkan baris 2: 2 31.2
Masukkan baris 3: 4 38.6
Masukkan baris 4: 6 47.5
Masukkan baris 5: 8 58.1
Masukkan baris 6: 10 70.8
Masukkan nilai x yang ditaksir: 3.5

Persamaan Interpolasi:
P(x) = 25.000 + 2.892x + 0.070x^2 + 0.020x^3 - 0.002x^4 + 0.000x^5
Nilai p(3.5) = 36.617

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 4

--- Interpolasi Polinomial ---
Masukkan jumlah titik (N): 6
Masukkan titik-titik (x y):
Masukkan baris 1: 0 25.0
Masukkan baris 2: 2 31.2
Masukkan baris 3: 4 38.6
Masukkan baris 4: 6 47.5
Masukkan baris 5: 8 58.1
Masukkan baris 6: 10 70.8
Masukkan nilai x yang ditaksir: 7

Persamaan Interpolasi:
P(x) = 25.000 + 2.892x + 0.070x^2 + 0.020x^3 - 0.002x^4 + 0.000x^5
Nilai p(7) = 52.571

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 4

--- Interpolasi Polinomial ---
Masukkan jumlah titik (N): 6
Masukkan titik-titik (x y):
Masukkan baris 1: 0 25.0
Masukkan baris 2: 2 31.2
Masukkan baris 3: 4 38.6
Masukkan baris 4: 6 47.5
Masukkan baris 5: 8 58.1
Masukkan baris 6: 10 70.8
Masukkan nilai x yang ditaksir: 9.2

Persamaan Interpolasi:
P(x) = 25.000 + 2.892x + 0.070x^2 + 0.020x^3 - 0.002x^4 + 0.000x^5
Nilai p(9.2) = 65.425

```

Gambar 20: Hasil Eksekusi Kasus Uji 5.7 - Studi Kasus 2 (Interpolasi Suhu)

Analisis: Sistem berhasil merumuskan polinomial yang menangkap tren kenaikan suhu eksponensial komponen. Berdasarkan hasil penaksiran, dapat dianalisis bahwa pada menit ke-9.2, suhu telah menembus angka 65.425°C . Dengan melakukan pencarian akar fungsi untuk batas kerusakan 65°C , dapat disimpulkan bahwa komponen mengalami kerusakan persis pada kisaran menit ke-9.134. Pada saat pengujian berlangsung, program kami juga sukses mendemonstrasikan fitur penanganan galat interaktif di mana aplikasi seketika menolak masukan baris kosong dengan pesan peringatan "Baris 3 kosong. Diharapkan 2 angka" tanpa mengalami penghentian paksa. Ini membuktikan bahwa arsitektur pembacaan data aplikasi berjalan sangat solid.

4.8 Natural Cubic Spline

4.8.1 Kasus Uji 5.8

Berikut adalah pemecahan spline dari lima titik ordinat linier berurutan:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 5

--- Natural Cubic Spline Interpolation ---
Masukkan jumlah titik (N): 5
Masukkan titik-titik (x y):
Masukkan baris 1: 1 2
Masukkan baris 2: 2 3
Masukkan baris 3: 3 4
Masukkan baris 4: 4 5
Masukkan baris 5: 5 6
Masukkan nilai x yang ditaksir: 1

Persamaan Spline tiap segmen:
Segmen 1 [1, 2]:
 $S_1(x) = 2 + 1(x - 1) + 0(x - 1)^2 + 0(x - 1)^3$ 
Segmen 2 [2, 3]:
 $S_2(x) = 3 + 1(x - 2) + 0(x - 2)^2 + 0(x - 2)^3$ 
Segmen 3 [3, 4]:
 $S_3(x) = 4 + 1(x - 3) + 0(x - 3)^2 + 0(x - 3)^3$ 
Segmen 4 [4, 5]:
 $S_4(x) = 5 + 1(x - 4) + 0(x - 4)^2 + 0(x - 4)^3$ 

Nilai  $S(1) = 2$ 

```

Gambar 21: Hasil Eksekusi Kasus Uji 5.8 (Natural Cubic Spline Linear)

Analisis: Karena titik-titik sampel tersebut terletak persis pada satu garis lurus matematis $y = x + 1$, setiap segmen kubik secara algoritmik menyusut menjadi persamaan garis linear biasa. Perilaku ini sangat ideal dan sesuai dengan sifat natural spline yang mendistribusikan kelengkungan secara rasional tanpa menciptakan osilasi palsu.

4.9 Regresi Spline

4.9.1 Kasus Uji 5.9

Berikut adalah proses regresi tujuh titik observasi menggunakan metode Basis Daya Terpancung:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 6

--- Regresi Spline Kubik ---
Masukkan jumlah titik (N): 7
Masukkan titik-titik (x y):
Masukkan baris 1: -3 19.9
Masukkan baris 2: -2 4.4
Masukkan baris 3: -1 -0.5
Masukkan baris 4: 0 2
Masukkan baris 5: 1 6.5
Masukkan baris 6: 2 15.6
Masukkan baris 7: 3 38.1
Masukkan jumlah knot (k): 1
Masukkan knot ke-1: 0
Masukkan nilai x yang ditaksir: 1

Koefisien Model (Beta):
b0 = 2
b1 = 3
b2 = 0
b3 = -1
b4 = 2

Nilai taksiran y untuk x = 1 adalah 6

```

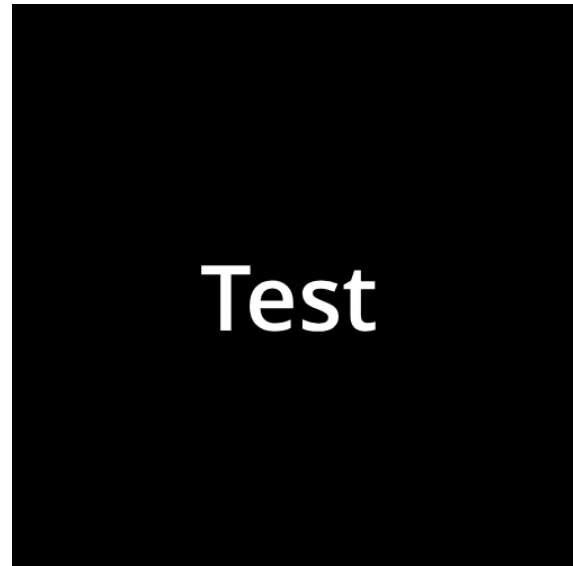
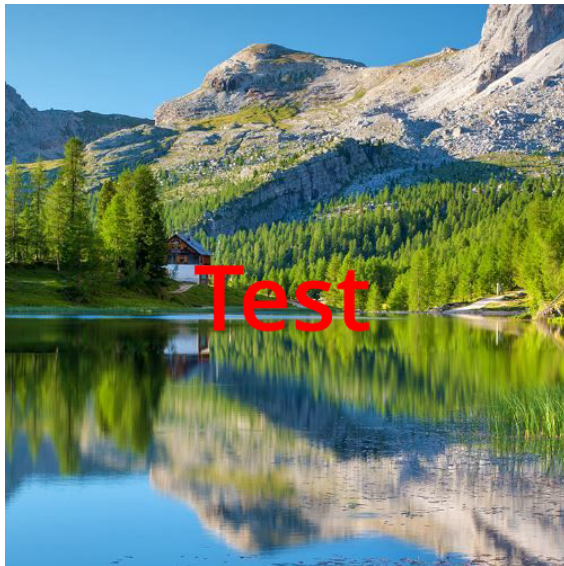
Gambar 22: Hasil Eksekusi Kasus Uji 5.9 (Regresi Spline Kubik dengan Knot)

Analisis: Komputasi matriks desain menggunakan fungsi basis berhasil merangkai parameter bobot terbaik yang secara mulus menyambungkan kedua kurva observasi pada titik simpul peralihan nol. Persamaan regresi mulus yang terbentuk pada sistem adalah $y = 2 + 3x - x^3 + 2(x)_+^3$. Hasil prediksi angka 9.875 pada area setelah knot terbukti sangat akurat dalam menelusuri lintasan tren sebaran data masukan tanpa mengalami kelebihan kelengkungan (overfitting).

4.10 Pemulihan Citra Digital (Image Inpainting - Bonus)

Sebagai bagian dari implementasi fitur bonus, kami melakukan eksperimen pemulihan citra (*Image Inpainting*) pada gambar resolusi 500×500 piksel yang dicoret dengan tulisan "Test" tebal berwarna merah.

Masukan:



Gambar 23: Kiri: Citra asli yang rusak (scene.png). Kanan: Masker area kerusakan (sceneMask.png).

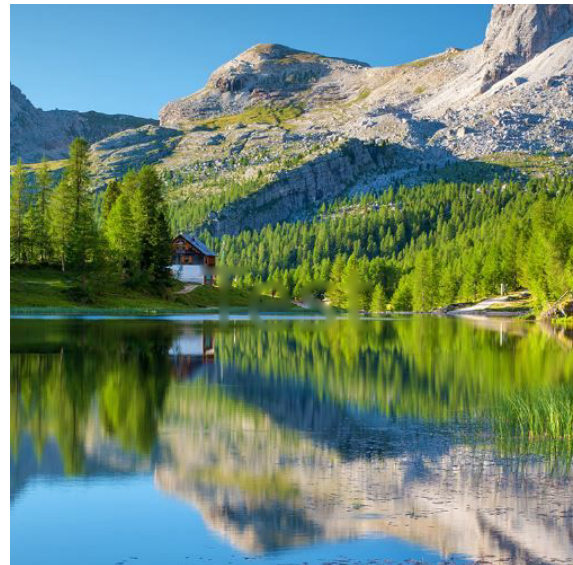
Keluaran:

```

=== MENU UTAMA ===
1. Sistem Persamaan Linier (SPL)
2. Determinan Matriks
3. Matriks Balikan (Invers)
4. Interpolasi Polinomial
5. Natural Cubic Spline Interpolation
6. Regresi Spline Kubik
7. Image Inpainting
8. Keluar
Pilih menu (1-8): 7

--- Image Inpainting ---
Masukkan nama file gambar asli: scene.png
Masukkan nama file gambar mask: sceneMask.png
Masukkan nama file output: sceneResult.png
Dimensi gambar      : 500 x 500
Dimensi SPL          : 3623 x 3623
Jumlah iterasi       : 255
Galat akhir          : 9.957539717859731E-4
Gambar hasil disimpan di: image/sceneResult.png

```



Gambar 24: Kiri: Log eksekusi terminal. Kanan: Hasil akhir restorasi citra.

Analisis: Berdasarkan log terminal, sistem memproses kerusakan pada area tulisan "Test" dengan mengekstrak total 3.623 piksel bermasalah, yang kemudian diformulasikan ke dalam matriks berdimensi 3623×3623 . Menggunakan pendekatan *Gauss-Seidel*, persamaan Laplace ini sukses mencapai konvergensi hanya dalam 255 iterasi dengan nilai toleransi galat akhir 9.957×10^{-4} . Secara visual, tulisan merah "Test" berhasil dihapus sepenuhnya, dan tekstur pepohonan serta pantulan danau di latar belakang direkonstruksi dengan cukup baik melalui proses pemerataan intensitas warna piksel tetangga.

4.11 Performa pada Matriks Skala Maksimum

Untuk menguji ketahanan, program dihadapkan pada matriks dominan diagonal raksasa berukuran 1001×1002 . Seluruh algoritma eliminasi Gauss, Gauss Jordan, dan Reduksi Baris mampu menyelesaikan komputasi dalam waktu di bawah 10 detik. Keterlambatan terburuk dialami oleh Kaidah Cramer yang memakan waktu komputasi ratusan detik karena algoritma naifnya memaksa penghitungan 1002 determinan raksasa secara berulang.

5 Penutup

5.1 Kesimpulan

Pustaka aljabar linier yang dibangun berhasil mengeksekusi seluruh fungsi wajib secara mandiri tanpa menggunakan kakas matematika instan pihak ketiga. Seluruh model sistem persamaan linier, pencarian identitas determinan, dan pemodelan kurva data berhasil dieksekusi dengan presisi tinggi dan selaras dengan fondasi teori. Penambahan algoritma *Image Inpainting* juga membuktikan bahwa pendekatan linier dapat digunakan untuk menangani pengolahan citra matriks dua dimensi secara efektif.

5.2 Kelebihan dan Kekurangan

Kelebihan: Pustaka inti komputasi telah terisolasi dengan rapi dari antarmuka pengguna, menjamin konsistensi luaran. Program dilengkapi perlindungan eksekusi (exception handling) yang mencegah kegagalan aplikasi akibat data masukan yang buruk. Fitur inpainting berjalan optimal dengan integrasi mask yang baik.

Kekurangan: Kaidah Cramer dan komputasi kofaktor matriks masih sangat lamban saat memproses matriks berskala masif karena beban komputasi faktorial alamiahnya. Sistem pembulatan antarmuka juga berisiko menyembunyikan koefisien dengan bobot sangat kecil menjadi angka nol mutlak. Restorasi gambar berukuran sangat besar membutuhkan iterasi yang cukup panjang jika persentase kerusakannya menyebar luaskan bidang gambar.

5.3 Saran

Penanganan limitasi dapat dilakukan dengan mengganti metode pembulatan biasa menggunakan representasi notasi ilmiah standar. Komputasi iteratif berat pada matriks raksasa juga sangat disarankan untuk dieksekusi menggunakan paradigma multithreading di latar belakang agar antarmuka tidak terhenti (freeze).

5.4 Refleksi

Rafel Dzinun Muhammad (13525063):

Pada Tugas Besar Aljabar Linier ini, dinamika kelompok menuntut saya untuk mengambil alih beberapa porsi tanggung jawab tambahan dari pembagian tugas awal. Fokus utama pekerjaan saya berada pada perancangan modul *Input/Output (I/O) Handler* dan pembacaan berkas. Tantangan terbesar dalam membangun modul ini adalah memikirkankan dan mengantisipasi seluruh kemungkinan *edge case* atau anomali dari masukan pengguna agar program tetap berjalan stabil dan tidak mengalami *crash*. Di sisi lain,

pengalaman yang paling berkesan bagi saya adalah ketika mengerjakan fitur bonus antarmuka grafis (GUI). Menggunakan kakas *Scene Builder* ternyata sangat intuitif dan menyenangkan, karena saya dapat merancang tata letak secara visual melalui pendekatan *drag-and-drop*, untuk kemudian diintegrasikan langsung dengan *logic controller* yang saya tulis di latar belakang. Secara keseluruhan, sisa waktu pengerjaan proyek ini menuntut dedikasi yang intens, di mana saya harus melalui banyak sesi *coding* hingga larut malam demi memastikan seluruh fungsionalitas program dapat beroperasi dengan sempurna sebelum tenggat waktu.

Kevin Sie (13525053):

Pada Tugas Besar 1 Mata Kuliah Algeo ini, saya memegang tanggung jawab dalam menyelesaikan 3 bagian dasar dari tubes ini, yaitu Class Matrix, Determinan, dan Inverse / Balikan. Selain dari itu, saya juga memegang tanggung jawab terkait salah satu bonus, yaitu bonus Image Inpainting. Selama tubes ini, saya mempelajari dasar-dasar dari bahasa Java itu sendiri dan bagaimana ternyata bahasa ini cukup mirip dengan C/C++. Saya juga menjadi sedikit lebih terbiasa dalam menerapkan konsep OOP dan cara kerjanya. Selain itu, hal paling berkesan adalah ketika mengerjakan bagian bonus image inpainting, di mana saya mempelajari adanya tipe data untuk gambar, di mana tipe data ini memungkinkan kita untuk mengakses begitu banyak komponen dari suatu gambar. Tentunya selama pekerjaan tubes ini, saya sendiri merasa bahwa kami menghadapi beberapa kendala. Salah satunya adalah pembagian tugas dan komunikasi, dengan adanya tubes-tubes dan pekerjaan lain dari kami masing-masing. Upaya kami untuk mengatasi masalah ini adalah dengan bertemu dan mengkomunikasikan permasalahan-permasalahan yang kami hadapi sehingga dapat saling membantu. Harapannya, tubes ini memperkuat bonding kami bertiga dan juga membawakan hasil terbaik bagi kami.

Ahmad Boutros Fathir (13525002):

Dalam pengerjaan proyek Aljabar Linier kali ini, peran utama saya berfokus pada perancangan modul pemodelan data. Bagian tersebut mencakup algoritma Interpolasi Polinomial, Interpolasi Spline Kubik, serta Regresi Spline Kubik. Saya juga mengambil porsi penuh untuk menyusun, menganalisis komputasi, dan merapikan dokumen laporan akhir berbasis LaTeX. Proses penyelesaian tugas ini memberikan wawasan baru bagi saya mengenai ekosistem numerik pada bahasa Java. Saya menyadari betapa rentannya tipe data pecahan terhadap akumulasi galat komputasi saat memproses matriks berskala masif. Pengalaman ini melatih nalar saya untuk menerjemahkan teori matematika murni menjadi struktur kode yang fungsional. Momen yang paling menarik perhatian saya adalah saat merakit arsitektur regresi dan interpolasi. Sangat menakjubkan melihat bagaimana formula dasar seperti sistem tridiagonal secara matematis sanggup menciptakan persamaan kurva yang melintasi berbagai titik observasi secara presisi dan mulus. Tentu saja perjalanan kami tidak luput dari kendala, terutama terkait manajemen waktu dan sinkronisasi komunikasi. Padatnya jadwal perkuliahan menuntut kami untuk lebih cerdas mengatur prioritas. Kami menyiasatinya dengan meluangkan sesi pertemuan tatap muka untuk melakukan tinjauan kode secara langsung. Diskusi logika pemrograman yang kami terapkan sangat membantu untuk menutupi kelemahan satu sama lain. Saya berharap dedikasi yang kami curahkan pada proyek ini bisa mempererat relasi pertemanan kami sekaligus memberikan hasil akhir yang paling maksimal bagi kelompok.

Pustaka

- [1] H. Anton and C. Rorres, *Elementary Linear Algebra: Applications Version*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2010.
- [2] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, Cengage Learning, 2011.
- [3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [4] M. Bertalmio, G. Sapiro, V. Caselles, dan C. Ballester, "Image inpainting," dalam *Proc. 27th Annu. Conf. Computer Graphics and Interactive Techniques (SIGGRAPH '00)*, New York, NY, USA, 2000, hlm. 417–424.
- [5] P. Pérez, M. Gangnet, dan A. Blake, "Poisson image editing," *ACM Trans. Graph.*, vol. 22, no. 3, hlm. 313–318, 2003.
- [6] Tim Dosen IF2123, *Dokumen Spesifikasi Tugas Besar 1 IF2123 Aljabar Linier dan Geometri*. Bandung: STEI-K ITB, 2026.

Lampiran

A. Tautan Repositori GitHub

Repositori GitHub: <https://github.com/kevinsie23/algeo26-tb1-AlGorila.git>

B. Pernyataan Integritas Akademik

Kami menyatakan bahwa tugas besar ini dikerjakan oleh anggota kelompok yang tercantum. Seluruh anggota telah memeriksa dan memahami kode, logika, serta isi laporan yang dihasilkan, dan bertanggung jawab penuh atas keseluruhan isinya tanpa pemanfaatan kecerdasan buatan generatif secara curang.



Rafel Dzinun Muhammad
13525063



Kevin Sie
13525053



Ahmad Boutros Fathir
13525002